CERTIFICATE

A

PROJECT REPORT ON

Project ID: GA08

PrepPro: A Scalable, Efficient and Self-hosted Quiz Portal for Academic Institution using Microservice based Architecture

Submitted for Partial Fulfillment of the Requirements for the Degree of Bachelor of Engineering in the Department of Computer Science and Engineering (AI & ML), Pimpri Chinchwad College of Engineering, Savitribai Phule Pune University, Pune

By

Rushikesh Biradar
Suyash Gadhave
Shailesh Kaliya
Koustubh Kulkarni

Dr. Sonal Gore	**Dr. A. D. Thakare**	**Dr. G. N. Kulkarni**
(Project Guide)	**(H.O.D)**	**(Director)**

Pimpri Chinchwad College of Engineering,
Savitribai Phule Pune University, Pune

2024-25

Scheme A

ACKNOWLEDGEMENT

We express our sincere thanks to our **Guide Dr. Sonal Gore** for her constant encouragement and support throughout our project, especially for the useful suggestions given during the project and for having laid down the foundation for the success of this work.

We would like to thank **Prof. Dr. Anuradha Thakare, Head of the CSE (AI & ML) Department,** for her unwavering support during the entire course of this project work. We are very grateful to our **Director, Prof. Dr. G.N. Kulkarni,** for providing us with an environment to complete our project successfully. We also thank all the staff members of our college and technicians for their help in making this project a success.

We also thank all the web committees for enriching us with their immense knowledge. Finally, we take this opportunity to extend our deep appreciation to our family and friends, for all that they meant to us during the crucial times of the completion of our project.

Name	Sign
Rushikesh Biradar	
Suyash Gadhave	
Shailesh Kaliya	
Koustubh Kulkarni	

ABSTRACT

In the current academic landscape, colleges often rely on outsourced platforms to conduct company-specific quizzes and tests aimed at placement preparation. These platforms, while functional, present several challenges including high costs, limited availability, lack of customization, and reduced control over data and administration. Our project addresses these issues by developing a comprehensive, on-premises quiz portal tailored for college use.

This system provides a **student-friendly interface** to practice **aptitude**, **core subject MCQs**, and **MSQs** (Multiple Select Questions), while offering a **powerful admin dashboard** for teachers to create, schedule, and manage tests effortlessly. It empowers faculty to **analyze individual and batch-level performance**, track student improvement over time, and generate actionable insights.

Design using a **microservices architecture**, the platform ensures **flexibility**, **scalability**, and **easy maintenance**. It supports **plug-and-play deployment**, enabling independent and modular setup within college infrastructure. With integrated features like **user authentication**, **Redis-based caching**, and **messaging for email notifications**, our solution provides a reliable, cost-effective, and fully **self-hosted alternative** to outsourced quiz systems, placing control back into the hands of educational institutions. In performance tests on moderate infrastructure, the system reliably supported up to **2000** concurrent users with an average response time of under **1000ms**. It demonstrated stable performance and scalability suitable for college-level deployments without requiring high-end servers.

KEYWORDS: Spring Boot, React JS, Microservices, Chatbot, Docker deployment, Kubernetes, JWT authentication

TABLE OF CONTENTS

Sr. No.	Title of Chapter	Page No.
01	**Introduction**	9
1.1	Motivation	9
1.2	Background	12
02	**Literature Survey**	12
2.1	Literature Survey Table	13
2.2	Summary of Literature Survey	15
2.3	Gap Identification / Common Findings from the Literature	16
2.4	Objectives	17
03	**Methodology**	20
3.1	Project Outline	20
3.2	Dataset Design	21
3.3	Architecture diagram/block diagrams and all UML diagrams	29
04	**Design & Implementation of Proposed System**	39
4.1	Project Cost Estimation	39
	4.1.1 Experimental Setups	39
	4.1.2 Procedures adopted	41
	4.1.3 Techniques developed	44
	4.1.4 Methodologies	46
	4.1.5 Algorithms developed	48
4.2	Important derivations/formulae	50
05	**Results and discussions**	54
5.1	Outcomes	54
5.2	Result analysis and validations	60
5.3	Screenshots	64
06	**Conclusion**	73
6.1	Conclusion	73
6.2	Future Work	74
6.3	Applications	75
	Appendix A: Details of paper publication: name of the conference / journal, comments of reviewers, certificate, and paper. Online published paper and certificates of participation in conference if any otherwise, draft paper. Or Patent filed documents, Project event participation certificates if any **Appendix B:** Plagiarism Report of project report.	76
	References	80

LIST OF ABBREVIATIONS

ABBREVIATION	ILLUSTRATION
MCQ	Multiple Choice Question
MSQ	Multiple Select Question
UI	User Interface
API	Application Programming Interface
DB	Database
REST	Representational State Transfer
JWT	JSON Web Token (for authentication)
Redis	Remote Dictionary Server

LIST OF FIGURES

FIGURE	ILLUSTRATION	PAGE NO.
3.1	Methodology Diagram	20
3.2	Dataset Design	21
3.3	SDLC Model	22
3.4	System Architecture Diagram	23
3.5	Activity Diagram (UML)	24
3.6	ER Diagram	25

LIST OF TABLES

TABLE	ILLUSTRATION	PAGE NO.
2.1	Literature Survey Summary	16
3.1	User Document Structure	22
3.2	Quiz Document Structure	24
3.3	Question Document Structure	25
3.4	QPFEntry Document Structure	27

CHAPTER 1

Chapter 01

INTRODUCTION

1.1 Motivation

In today's competitive placement environment, colleges strive to provide students with high-quality resources to prepare for company-specific recruitment tests, including aptitude, technical MCQs, and MSQs. However, the current platforms used for conducting such quizzes are often **outsourced** and come with several **critical limitations** that hinder their effectiveness and accessibility.

One of the major issues is **availability**. Students frequently face **downtime**, **server lags**, and **slow response times**, especially during peak hours or mass testing sessions. This not only disrupts the learning experience but also affects the reliability of practice sessions. Moreover, the **quality of service** is inconsistent, with poor UI/UX and limited features for both students and teachers.

Another key challenge lies in **ownership and control**. Outsourced systems offer **little to no flexibility** for colleges to customize the platform according to their specific requirements. Faculty members lack administrative autonomy and struggle to track student progress, manage test content, and resolve queries quickly. In many cases, **support for high-volume concurrent users is weak**, resulting in failed sessions or delayed feedback.

Costing is also a significant concern. These third-party services often demand recurring subscriptions or usage-based fees, which can strain the budgets of educational institutions. Furthermore, **performance bottlenecks**, **inability to scale on demand**, and **limited analytics tools** reduce the overall value these platforms provide.

Our motivation is to **overcome these limitations** by offering a **reliable**, **cost-effective**, and **easily deployable** in-house quiz platform. The goal is to ensure **high availability**, **low latency**, and **robust performance**, even during high-load scenarios. By building a **self-hosted, microservice-based system**, we give colleges full **control over the platform**, enabling them to customize test structures, scale based on demand, resolve queries faster, and continuously improve based on real performance data.

This platform is designed not just as a tool for testing, but as a **strategic academic asset**—empowering both students and educators with seamless, data-driven test management and preparation experiences.

1.2 Background

With the increasing emphasis on campus placements, academic institutions are under pressure to provide effective tools for student preparation. Placement drives typically involve a variety of assessments such as **aptitude tests**, **domain-specific MCQs**, and **company-specific quizzes**. To address this, many colleges currently rely on **third-party or outsourced quiz platforms**.

While these platforms offer basic functionality, they are often plagued with issues such as **frequent downtime**, **poor scalability**, **limited customization**, and **high recurring costs**. Students frequently report problems like **test lags**, **slow loading times**, and **sudden disconnections**, which can be demotivating during high-stakes preparation. Teachers, on the other hand, have **restricted access** to performance data and limited administrative control, making it difficult to tailor content or analyze student progress effectively.

Moreover, with outsourced platforms, institutions often face **data security concerns** and **lack of ownership** over their content and user data. These drawbacks collectively hinder the overall effectiveness of placement training efforts and diminish the learning experience for students.

In response to these challenges, there is a clear need for a **dedicated, college-owned quiz platform** that is **reliable**, **scalable**, and **custom-built** for academic environments. By adopting a **microservices-based architecture** with **on-premises, plug-and-play deployment**, colleges can regain control, ensure better system performance, and create a personalized and impactful testing ecosystem.

This project is born out of the desire to **empower institutions** with a tool that aligns with their academic goals while eliminating the operational and technical drawbacks of outsourced solutions.

CHAPTER 2

Chapter 2

LITERATURE SURVEY

2.1 Literature Review

To guide the development of PrepPro, a detailed literature review was conducted, exploring existing solutions and academic research in the domain of online quiz systems, exam security, and scalable educational platforms. These papers provided valuable insights into best practices and informed decisions regarding design, architecture, and feature implementation.

2.1.1. Design and Implementation of an Online Examination System [1]

- A 2021 paper on the design and implementation of an online examination system developed using Microsoft Visual Studio and SQL for university students in Iraq. The system included essential features like student login, random question generation, and role-based access control.
- The primary goal was to reduce human intervention in the evaluation process by automating various aspects of the exam. Despite these advancements, the system struggled with scalability, which limits its application to smaller educational institutions.

2.1.2. AI-based Online Proctoring System with YOLO-v3 & MMOD CNN [2]

- A 2023 study discusses an AI-based proctoring system using YOLO-v3 and MMOD CNN algorithms for real-time monitoring and detection of academic dishonesty during online exams. The system uses camera feeds to detect suspicious behaviors like cheating and impersonation.
- While effective in behavior analysis, the system raises concerns about privacy, with potential for false positives and biased results. The authors suggest that combining multiple detection modalities could improve the system’s accuracy and fairness.

2.1.3. Real-Time Quiz Management Platform with Analytics [3]

- A 2021 article presents the creation of a real-time quiz application using Node.js, WebSockets, and MongoDB. The system offers interactive quiz features, including real-time scoring, live question updates, and analytics dashboards.
- Although it performs well in smaller classrooms, it faces challenges when scaling for high-concurrency scenarios, which can affect system performance under heavy loads.

2.1.4. Microservices Architecture for Scalable Online Education Platforms [4]

- A 2022 paper explores using microservices architecture, specifically with Spring Boot and Kubernetes, to build scalable and resilient online education platforms. This architecture enables modular design and easy scaling of components like authentication, quiz management, and reporting.
- The use of Docker and Kubernetes ensures fault tolerance and automatic scaling, making the system suitable for institutions with large user bases, including support for thousands of concurrent users.

2.1.5. Automating Excel Reports with Python [5]

- A 2020 blog post explores how to automate the generation of Excel reports using Python. The tutorial emphasizes the use of libraries like Pandas and OpenPyXL to automate the process of data manipulation, report generation, and formatting.
- This approach simplifies the task of creating and maintaining detailed Excel reports and is particularly useful for large datasets and frequent report generation, reducing manual effort and increasing efficiency.

Literature review is summarized in the following table 2.1.

Table 2.1: Literature Survey Summary

Sr. No.	Title	Year	Publisher	Methodology	Remarks
1.	Design and Implementation of an Online Examination System	2021	Scientific. net	1. Developed a web-based examination system using PHP and MySQL. 2. Modular design covering student login, question paper generation, and evaluation. 3. Used role-based access control to manage students and instructors. 4. Implemented features like random question selection and timer-based submission.	1. Enhanced exam security and reduced human effort in evaluation. 2. User-friendly interface for students and instructors. 3. Lacks real-time scalability and concurrency handling for large-scale exams. 4. Suitable for small institutions or pilot projects.
2.	AI-based Online Proctoring	2023	Research Gate	1. Reviewed existing AI-based proctoring tools	1. Proctoring tools improve academic

	System with YOLO-v3 & MMOD CNN			with webcam and microphone integration. 2. Face detection using CNN, anomaly behavior detection via keystroke and eye movement tracking. 3. Compared YOLO-V3 and MMOD CNN for object detection in live video feeds. 4. Emphasis on reducing false positives in cheating detection.	integrity during online exams. 2. Raises data privacy and ethical concerns. 3. Suggests combining multiple AI models for better accuracy and fairness. 4. Recommended adoption with clear privacy policies and user consent.
3.	Real-Time Quiz Management Platform with Analytics	2021	Medium	1. Built with Node.js backend and WebSocket-based communication for real-time interaction. 2. MongoDB used to store quiz results, user behavior, and session logs. 3. Integrated dashboard for analytics (scores, completion time, engagement). 4. Admins can schedule, start, and monitor live quizzes.	1. Real-time interaction improved teacher control during quizzes. 2. Analytics provided valuable feedback on question difficulty and student performance. 3. Performance drops noted during very high concurrent usage — needs optimization. 4. Best suited for mid-size online classrooms.
4.	Microservices Architecture for Scalable Online Education Platforms	2022	Medium	1. Cloud-based architecture using Kubernetes and Docker for containerized deployment. 2. Load balancer used to distribute traffic across multiple quiz service nodes. 3. MongoDB used for fast and scalable data storage.	1. Efficient performance under heavy load. 2. High availability ensured through auto-scaling of services. 3. Supports modular microservice-based extensions. 4. Recommended for deployment in educational

				4. Simulated with up to 2000 concurrent users during testing.	institutes with large student bases.
5.	Automating Excel Reports with Python	2020	Plotly	1. Utilized Python libraries like Pandas and OpenPyXL to automate the creation of detailed Excel reports. 2. Processed large datasets from quizzes and student responses. 3. Simplified data manipulation, formatting, and presentation of results. 4. Eliminated manual report generation, improving efficiency.	1. Greatly reduces manual reporting efforts and improves performance tracking for educators. 2. Streamlined the process for large datasets and frequent report generation. 3. Reduces human error in report creation. 4. Ideal for large-scale result analysis and record-keeping.

2.2 Summary of Literature Survey

- **Web-based and Real-Time Quiz Platforms**: Several studies explore the development of online quiz systems that employ web technologies to deliver quizzes, track progress, and provide real-time updates. Key aspects like session management, real-time interactions, and user authentication are consistently emphasized for enhanced user engagement and learning outcomes.

- **AI and Proctoring in Examination Systems**: AI-based proctoring systems are becoming increasingly popular for ensuring academic integrity during online exams. These systems utilize webcam and microphone input to detect suspicious behaviors, improving security. However, privacy and ethical concerns persist, and systems need to refine accuracy through multi-modal detection and better privacy protocols.

- **Scalability and Microservices Architecture**: A common theme in many studies is the use of cloud-native, microservice-based systems that offer high scalability and performance. These architectures use Docker and Kubernetes to ensure fault tolerance, high availability, and ease of deployment. This approach is particularly beneficial for institutions with large user bases.

- **Real-Time Analytics and Management:** The integration of real-time analytics into quiz systems allows instructors to monitor performance dynamically. Many platforms utilize databases like MongoDB to store quiz data and provide insights into student performance. However, scalability issues arise when handling a large number of concurrent users, particularly for more complex systems.

- **Interactive UI and Performance Analytics**: Platforms with intuitive UIs, session tracking, and built-in analytics empower educators to monitor student progress, analyze performance, and refine assessment strategies effectively.

- **Automated Report Generation**: Several systems have incorporated Python and Excel-based tools for automated report generation, allowing educators to extract insights efficiently and save time on manual tasks.

2.3 Gap Identification / Common Findings from Literature Survey

- **Scalability Challenges:**
  Many existing quiz systems are not optimized for high concurrency. Systems often experience performance degradation with thousands of concurrent users, which is critical for campus-wide exams.

- **Limited Real-Time Capabilities:**
  Real-time updates, such as live score updates, timer sync, and quiz status tracking, are often under-implemented or inefficient at scale.

- **Incomplete Proctoring Solutions:**
  Current AI-based proctoring lacks robustness across diverse hardware setups and often raises privacy and ethical concerns. Few systems integrate multi-modal detection (face, audio, activity monitoring) cohesively.

- **Weak Integration of Analytics:**
  Although analytics tools exist, they are rarely embedded deeply into the quiz platform. There is a gap in generating actionable, real-time insights directly usable by faculty and admins.

- **Usability and Admin Burden:**
  Manual efforts in uploading students or adding questions are time-consuming. Systems lack bulk upload support and user-friendly administrative interfaces, increasing operational overhead.

- **Absence of Offline/Local Deployments:**
  Very few platforms offer support for local network deployments — an essential feature in low-connectivity regions or during outages.

2.4 Problem Statement and Objectives

To develop a scalable, efficient, user-friendly and self-hosted quiz platform tailored for academic institutions to support various test-related tasks using microservice based architecture.

Core Functional Objectives

- To design a clean and intuitive user interface that enhances the experience for both students and teachers.

- To ensure high availability and performance through caching, load balancing, and optimized database queries.
- To support more than 1000 concurrent users by implementing efficient session and socket handling.
- To enable horizontal scalability using a microservices-based architecture with Docker.
- To build a cost-effective solution by utilizing open-source technologies and enabling self-hosted deployments for institutions.
- To simplify deployment and system management using Docker, Docker Network, and integration with system services on Ubuntu.
- To offer compatibility with online or local campus networks for uninterrupted functionality during exams.
- To implement secure authentication and authorization using JWT tokens and role-based access control.
- To automate performance reporting and provide administrators with advanced quiz management and bulk upload capabilities.

CHAPTER 3

Chapter 3

METHODOLOGY

3.1 Project Outline

The **PrepPro Quiz Platform** is a scalable, high-performance web application designed to streamline the creation, delivery, and management of quizzes for academic institutions, particularly targeting placement preparation. The platform emphasizes simplicity, high availability, and concurrent user support.

This project uses a modern microservices-based architecture with key components developed using **Spring Boot (Java)** for backend APIs, **React.js** for a responsive and intuitive frontend, and **Data Transformation** for handling data processing tasks like report generation. The database layer uses **MongoDB**, ensuring reliable data storage, and Docker is used for containerization and deployment across different environments.

Key features include:

- **Role-based authentication and authorization** to securely manage access.
- **Comprehensive admin panel** to manage students, quizzes, schedules, and performance metrics.
- **Proctored quiz mode** for simulating real exam conditions.
- **Automated Excel report generation** for performance analysis.
- **Offline and local network compatibility** for uninterrupted exams.

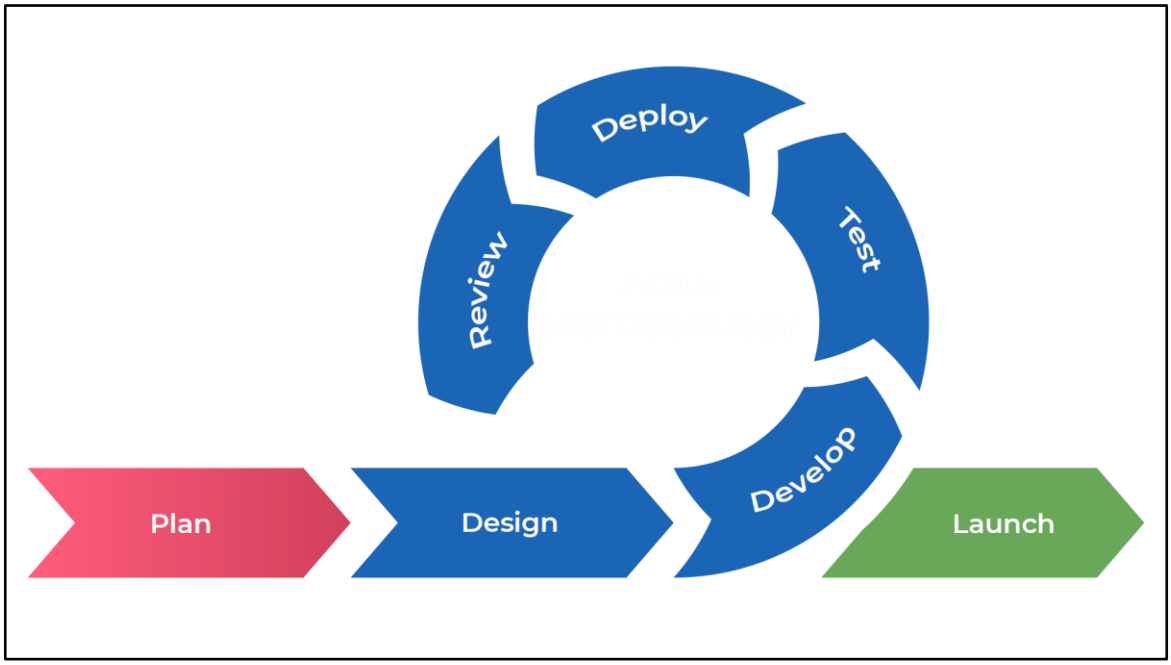


Fig. 3.1 *SDLC Diagram*

3.2 Dataset/Database Design

The backend system incorporates a well-structured and scalable schema with the following core collections:

- **Users Collection**

  Purpose:

  - Stores user details and tracks quizzes attempted by the user. Implements role-based access.

  Fields are mentioned in Table 3.1

Table 3.1 *User Document Structure*

Field Name	**Data Type**	**Description**
userId	String	Unique identifier for the user.
name	String	Full name of the user.
email	String	User's email address (used for login).
password	String	Encrypted password.
attendedQuizzes	Map<String, LocalDateTime>	Tracks quiz IDs and timestamps of quizzes attended.

createdAt	LocalDateTime	Timestamp when the user account was created.
role	int	User role: `0` for Admin, `1`


```
_id: "165a1430-f1ec-41c0-bb32-2dc963b2659b"
name : "Admin"
email : "Group08.admin@gmail.com"
password : "$2a$10$xw6iiBfpVo91lSVfN.0jR.RzinqfHZUvtP3/uHRn.ZCPwcxMvqtKa"
▸ attendedQuizzes : Object
createdAt : 2025-04-18T18:34:05.989+00:00
role : 0
_class : "com.quizz.user.User.Model.UserModel"
```


Json Representation of User Document:


```
{
  "name": "Admin",
  "email": "Group08.admin@gmail.com",
  "password": "$2a$10$xw6iiBfpVo91lSVfN.0jR.RzinqfHZUvtP3/uHRn.ZCPwcxMvqtKa",
  "attendedQuizzes": {
    "4c43d72c-d898-4303-b1d4-4d2560e1db13": {
      "$date": "2025-04-20T21:24:50.022Z"
    }
  },
  "createdAt": {
    "$date": "2025-04-18T18:34:05.989Z"
  },
  "role": 0,
  "_class": "com.quizz.user.User.Model.UserModel"
}
```


- **Quiz Collection**
  Purpose:
  - Stores metadata related to quizzes such as title, creator, duration, and state.

  Fields are mentioned in Table 3.2

Table 3.2 *Quiz Document Structure*

Field Name	**Data Type**	**Description**

quizId	String	Unique ID for the quiz.
title	String	Title or name of the quiz.
companyName	String	Associated company or subject name.
quizTime	int	Duration of the quiz in minutes.
numQuestions	int	Number of questions in the quiz.
createdAt	LocalDateTime	Timestamp when the quiz was created.
createdBy	String	Created by admin.
state	int	State of the quiz: `0` for Draft, `1` for Active.

```
_id: "8242eb4d-9f9a-4c64-89c3-6a172a388869"
title : "quiz 1"
companyName : "tcs"
quizTime : 0
numQuestions : 0
createdAt : 2025-02-07T20:49:46.148+00:00
createdBy : "ADMIN"
_class : "com.quizz.quiz.quiz.Model.QuizModel"
```

Json Representation of Quiz Document:


```
{
  "title": "quiz 1",
  "companyName": "tcs",
  "quizTime": 0,
  "numQuestions": 0,
  "createdAt": {
    "$date": "2025-02-07T20:49:46.148Z"
  },
  "createdBy": "ADMIN",
  "_class": "com.quizz.quiz.quiz.Model.QuizModel"
}
```

- **Questions Collection**

  Purpose:

  - Stores all questions associated with quizzes, including their options and correct answers.

  Fields are mentioned in Table 3.3

Table 3.3 *Question Document Structure*

Field Name	**Data Type**	**Description**
questionId	String	Unique identifier for the question.
quizId	String	ID of the quiz this question belongs to.
questionCategory	int	Category/type of question (MCQ, MSQ).
questionContent	String	The question text/content.
options	Map<String, String>	Map of option labels and their respective text (e.g., "A" → "Option 1").
correctOptions	List<String>	List of correct option keys (e.g., ["A", "C"]).
createdAt	LocalDateTime	Timestamp when the question was created.
createdBy	String	User role of the creator.

```
_id: "5a7dea96-8ad3-47ef-a809-3622be614a0d"
quizId : "8242eb4d-9f9a-4c64-89c3-6a172a388869"
questionCategory : 3
questionContent : ""What is HTTP?"
▾ options : Object
    A : "A protocol"
    B : "A programming language"
    C : "An OS"
    D : "A database"
▾ correctOptions : Array (1)
    0: "A"
createdAt : 2025-02-08T03:43:52.446+00:00
createdBy : "ADMIN"
_class : "com.quizz.question.question.Model.QuestionModel"
```


Json Representation of Question Document:


```
{
  "quizId": "8242eb4d-9f9a-4c64-89c3-6a172a388869",
  "questionCategory": 3,
  "questionContent": "\"What is HTTP?",
  "options": {
    "A": "A protocol",
    "B": "A programming language",
    "C": "An OS",
    "D": "A database"
  },
  "correctOptions": [
    "A"
  ],
  "createdAt": {
    "$date": "2025-02-08T03:43:52.446Z"
  },
  "createdBy": "ADMIN",
  "_class": "com.quizz.question.question.Model.QuestionModel"
}
```


- **QPFEntry Collections**
  Purpose:
  - Tracks each student's quiz performance and attempts, storing analytical metrics.

  Fields are mentioned in Table 3.4

Table 3.4 *QPFEntry Document Structure*

Field Name	**Data Type**	**Description**
qpfEntryId	UserQuizId	Composite key containing userId and quizId.
maxScore	int	Maximum score that can be achieved.
gainedScore	int	Score obtained by the user.
totalTime	int	Total quiz time allowed (in minutes).
timeSpent	int	Time actually spent by the student.
totalQuestions	int	Total number of questions in the quiz.
correctAttempted	int	Number of questions answered correctly.
incorrectAttempted	int	Number of questions answered incorrectly.
unAttempted	int	Number of questions not attempted.
lastScore	int	Score obtained in the last attempt.
currentAttemptTime	LocalTime	Time of the current attempt.
lastAttemptTime	LocalTime	Time of the last attempt.

```
▸ _id: Object
  maxScore : 9
  gainedScore : 0
  totalTime : 300
  timeSpent : 24
  totalQuestions : 9
  correctAttempted : 0
  incorrectAttempted : 1
  unAttempted : 8
  lastScore : -1
  currentAttemptTime : 2025-04-21T06:46:30.901+00:00
  _class : "com.quizz.quizProfile.qpfEntry.Model.QPFEntry"
  lastAttemptTime : 2025-04-21T06:09:20.266+00:00
```


Json Representation of QPFEntryDocument:


```
{
 "maxScore": 9,
 "gainedScore": 0,
 "totalTime": 300,
 "timeSpent": 24,
 "totalQuestions": 9,
 "correctAttempted": 0,
 "incorrectAttempted": 1,
 "unAttempted": 8,
 "lastScore": -1,
 "currentAttemptTime": {
  "$date": "2025-04-21T06:46:30.901Z"
 },
 "_class": "com.quizz.quizProfile.qpfEntry.Model.QPFEntry",
 "lastAttemptTime": {
  "$date": "2025-04-21T06:09:20.266Z"
 }
}
```

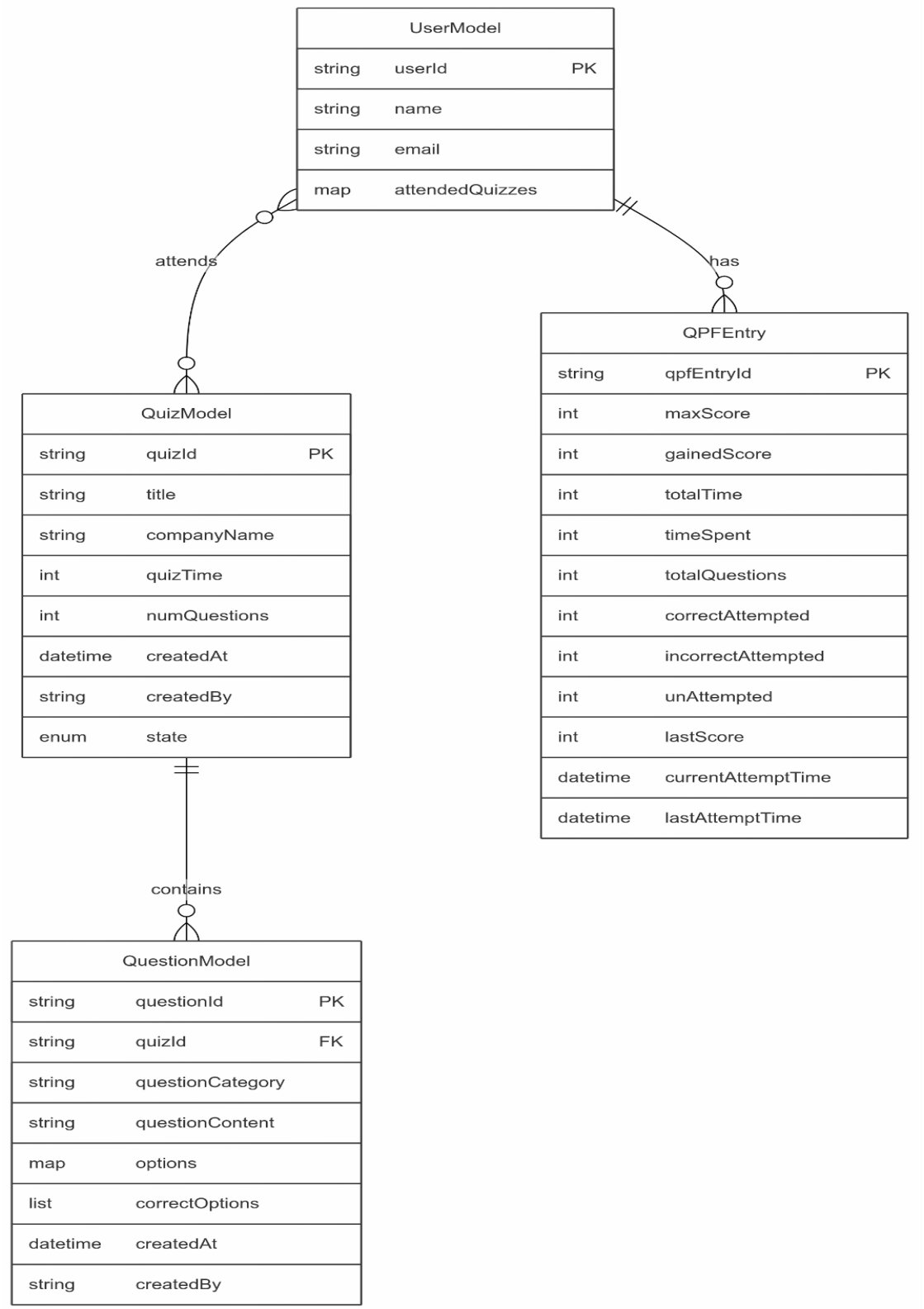


Fig. 3.2 *Dataset Design*

3.3 Architecture diagram/block diagrams and all UML diagrams

3.3.1 System Architecture Design

The architecture diagram below illustrates the microservices-based design of the PrepPro Quiz Portal. It showcases how core services like User, Quiz, Quiz Profile (QPF), and Email are modularized and managed independently, promoting scalability and maintainability. Central to this setup is the API Gateway, which acts as the single entry point for client requests, while the Eureka Service Registry handles service discovery, enabling seamless communication between distributed components. This architecture ensures flexibility, efficient load distribution, and a robust framework suitable for real-world academic environments.

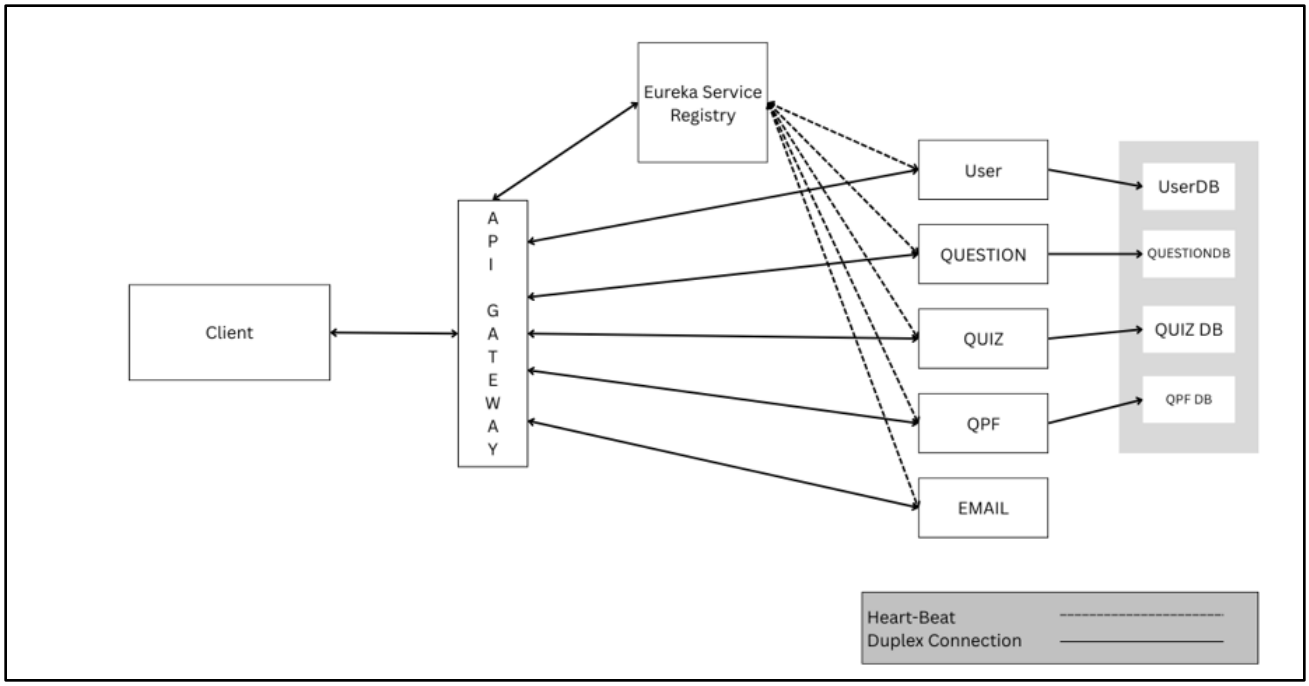


Fig. 3.3 *System Architecture Diagram*

- **Client**

At the top layer, we have a **ReactJS-based frontend** that serves as the main user interface. It provides role-specific dashboards for students, faculty, and administrators. The interface is designed to be responsive, user-friendly, and simulates a real-world test-taking environment. It communicates with backend services through REST APIs.

- **API Gateway (Spring Cloud Gateway)**

The frontend routes all API requests to the **Spring Cloud Gateway**, which acts as a **single entry point** to the system. It handles:

- **Routing** to appropriate microservices
- **Load balancing**
- **Security filtering** (like JWT validation)

This ensures decoupling between the frontend and individual services.

- **Eureka Service Registry**

Eureka functions as the **service discovery component** in this architecture. All backend microservices register themselves with Eureka, which enables dynamic discovery and interaction among services. It eliminates the need for hardcoding service locations and ensures **scalable and fault-tolerant communication**.

- **User Service**
  - Handles user registration, login, role assignment, and profile management.
  - Communicates with **User DB** for storing user data securely.

- **Quiz Service**
  - Responsible for quiz creation, scheduling, result storage, and time tracking.
  - Interfaces with **Quiz DB**.

- **QPF Service (Quiz Participation & Flow Service)**
  - Manages real-time quiz participation, answer submission, and section transitions.
  - Integrates browser cache and recovery mechanism.
  - Stores quiz attempt data in **QPF DB**.

- **Question Service**
  - Provides APIs to manage the question bank including MCQs and MSQs.
  - Offers randomization and topic-wise question retrieval.
  - Stores data in **Question DB**.

- **Email Service**
  - Sends email notifications for registration, quiz results, and admin communications.

  - Interacts with an external **SMTP server**.
  - Triggered via Redis-based messaging for decoupled notification handling.
- **Databases and External Services**

Each service maintains its own **dedicated database**, ensuring data isolation and reducing tight coupling. The **Email Service** connects to an external **SMTP server**, maintaining outbound communication through standard mail protocols.

3.3.2 Activity Diagram

The following activity diagram illustrates the end-to-end flow of actions performed by both Admin and Student within the PrepPro Quiz Portal. It outlines the key functionalities for each user role—from quiz creation and student onboarding on the admin side to dashboard navigation, quiz participation, and result viewing on the student side. The diagram clearly delineates the responsibilities and interactions, highlighting how performance data is evaluated and presented to the respective users, ensuring a smooth and structured user journey across the platform.

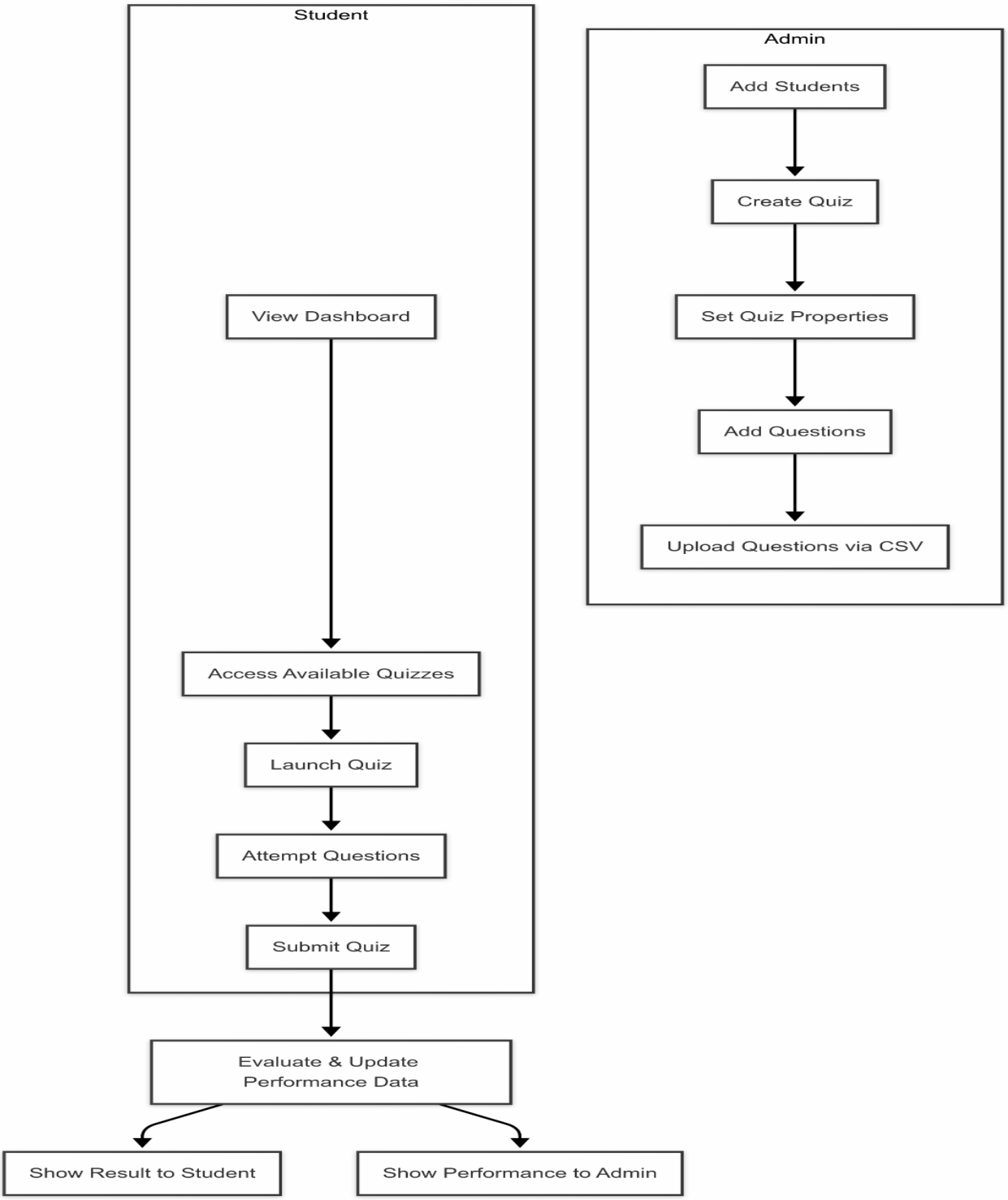


Fig. 3.4 *Activity Diagram*

- **Student Workflow (Left Lane)**

This swimlane captures the process a student follows from logging into the platform to viewing results after submitting a quiz.

- **View Dashboard:** The student logs in and lands on a personalized dashboard displaying quiz status, performance summaries, and available quizzes.

- **Access Available Quizzes:** The student navigates to the list of active or upcoming quizzes assigned to them.

- **Launch Quiz:** The student selects a quiz to begin. The platform initializes the quiz session with settings such as timer, question order, and navigation rules.

- **Attempt Questions:** Students go through the quiz, answering multiple-choice or multiple-select questions as defined by the faculty.

- **Submit Quiz:** Once completed, the student submits their responses. This step also allows for auto-submission if the timer runs out.

- **Evaluate & Update Performance Data:** The system automatically evaluates the answers and updates performance metrics based on correctness, completion, and time.

- **Show Result to Student:** A result summary is displayed, giving the student instant feedback on their performance.

- **Show Performance to Admin:** Simultaneously, the performance data is logged and made accessible to the admin for analysis and report generation.

- **Admin Workflow (Right Lane)**

The admin side reflects the activities of a faculty member or test administrator preparing and managing quizzes.

- **Add Students:** Admins can add individual students or upload student data in bulk via CSV. This step ensures users are on boarded into the system.

- **Create Quiz:** The admin initiates the creation of a new quiz, entering basic information like quiz name, duration, and subject.

- **Set Quiz Properties:** Configurations such as total time, question type (MCQ/MSQ), number of questions, and proctoring rules are defined.

- **Add Questions:** Questions can be manually added through the interface or selected from an existing question bank.

- **Upload Questions via CSV:** For efficiency, admins can upload multiple questions at once in a pre-defined CSV format, enabling rapid setup of assessments.

3.3.3 Deployment Diagram

The deployment diagram below demonstrates how the various microservices of the PrepPro Quiz Portal are orchestrated and deployed using Docker containers on a single host machine. Each service, including the API Gateway, UI, and domain-specific microservices like User, Quiz, and Email, is assigned a unique port for communication. A Docker network facilitates internal communication between services, while specific ports (such as 8090 for UI) are exposed to the host machine, allowing end users to interact with the system over the internet. This containerized setup ensures portability, scalability, and ease of deployment across environments.

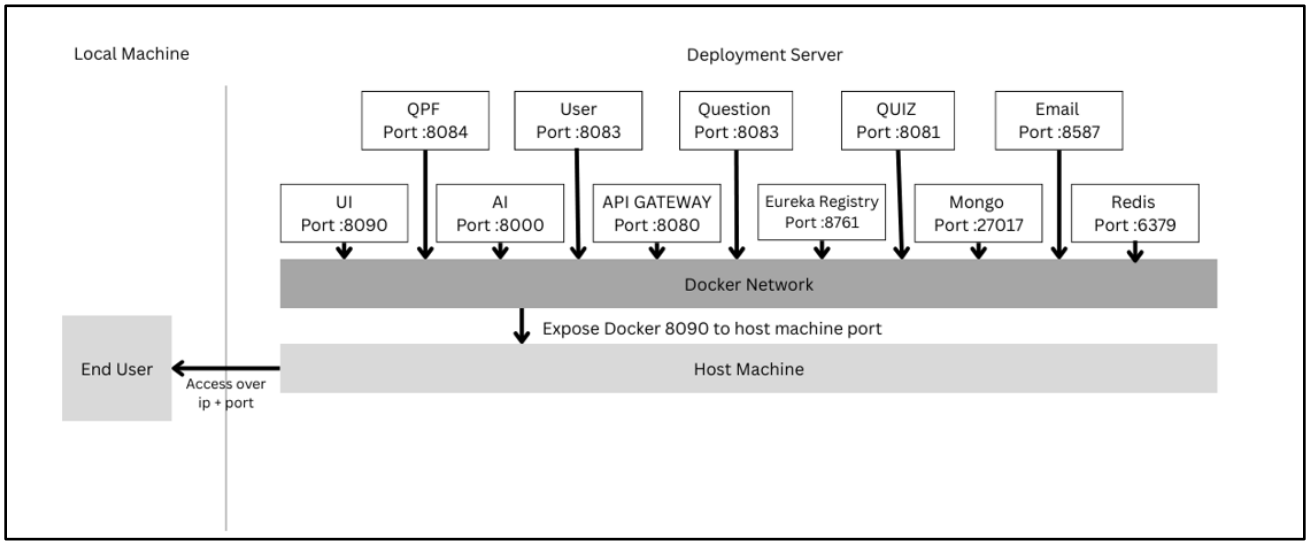


Fig. 3.5 *Deployment Diagram*

- **Docker Network**

All components are deployed inside a **Docker Network**, allowing inter-container communication by service name. This encapsulation improves portability, simplifies orchestration, and ensures environmental consistency across development, testing, and production.

- **UI Container**
  - **Port: 8080 (Host Exposed)**

- Hosts the **ReactJS frontend**.
- Exposes the user interface to the external world.
- Communicates with the API-Gateway for all backend interactions.

- **API Gateway (Port: 8090)**
  - Serves as the **central access point** for all frontend requests.
  - Routes incoming requests to the appropriate microservice.
  - Performs security filtering, routing, and rate-limiting.
  - Simplifies frontend-backend communication and hides service complexity.

- **Microservices and their Deployment**

Each service runs as a separate container with its own port and functionality:

- **QUIZ-SERVICE (Port: 8081):**
  Handles quiz creation, scheduling, and metadata management. Interacts with MongoDB to store quiz-related data.

- **QUESTION-SERVICE (Port: 8082):**
  Manages the question bank including MCQs and MSQs. Also communicates with MongoDB for persistent question data storage.

- **QPENTRY-SERVICE (Port: 8084):**
  Handles quiz participation flow, answer submissions, timer tracking, and implements **browser cache-based recovery mechanism**. Central to quiz-taking sessions.

- **USER-SERVICE (Port: 8083):**
  Manages user registration, login, JWT authentication, and role assignments. Stores user data in MongoDB and communicates with the Email Service for notifications.

- **EMAIL-SERVICE (Port: 8587):**
  Handles all email-based communications such as registration confirmations and quiz

notifications. Uses an external **SMTP server (Port: 587)** for sending emails. Triggered asynchronously via Redis-based messaging.

5. Database Layer

- **MongoDB (Port: 27017):**
  A single MongoDB instance is used for simplicity, hosting collections for quizzes, users, questions, and quiz participation logs.

6. Service Registry (Eureka) (Port: 8761)

- Each microservice registers itself with the **Eureka Service Registry**, enabling **service discovery** and **load balancing**.

- Ensures resilience by allowing dynamic scaling and self-healing of services.

7. Inter-Service Communication

- All services communicate with one another using **HTTP REST APIs** via the Docker network.
- For inter-service communication, **Spring Cloud OpenFeign** library is used to invoke web services to a client.
- The arrows indicate the communication pathways, including service-to-service calls and direct database access.

CHAPTER 4

Chapter 4

DESIGN & IMPLEMENTATION OF PROPOSED SYSTEM

This section outlines the design and implementation phases of the proposed quiz platform, describing the project's technical setup, procedures, methodologies, and key algorithms developed to ensure its success.

4.1 Project Cost Estimation

4.1.1 Experimental Setups

To ensure the robustness, reliability, and scalability of the quiz platform, multiple experimental setups were executed. These setups simulate real-world usage patterns in academic environments and test the system under various stress and load conditions.

1. **Server Configuration**

Table 4.1 *Hardware Configuration Setup*

Hardware Configuration	Specs
Student Test Devices	Laptops with **8GB/16GB RAM**, **6-core CPUs**
Faculty/Admin Devices	Laptops/Desktops with **16GB RAM**, **8+ cores**
Server (Local Testing)	Ubuntu server VM with **16GB RAM**, **8-core CPU**
Server (High Load Simulation)	Cloud VM with **64-core processor**, **128GB RAM**, Ubuntu 22.04 LTS

The system was tested on a range of hardware setups to verify deployment flexibility and resource usage efficiency. All servers were configured to support Docker and systemctl-based service management.

2. **Software Stack**

- Initially experimented with the **MERN stack**:

  - Used **MongoDB** as a NoSQL database, **Express.js** and **Node.js** for backend APIs, and **React.js** for the frontend.
  - Quick to build but limited scalability and heavy backend load.
- Switched to **Spring Boot + React.js** stack:
  - Spring Boot provided **robust backend support**, **faster API performance**, and **enterprise-grade scalability**.
  - Retained **React.js** for frontend due to its component-based flexibility.
- Choose **MongoDB** over MySQL due to:
  - **Schema-less nature** – allowed dynamic document structures for quizzes.
  - **Horizontal scalability** – supported high read/write throughput.
  - **Faster performance** with large amounts of unstructured data.

3. Microservices Setup

- Initially followed a **monolithic architecture**:
  - All functionalities like user management, quiz handling, and question storage were in a single codebase.
  - Faced difficulties in scaling individual modules and managing codebase.
- Shifted to a **Microservices Architecture**:
  - Divided application into **independent services** like:
    - **User Service**
    - **Quiz Service**
    - **Question Service**
    - **QPFEntry Service**
  - Enabled **scaling specific services** based on load (e.g., quiz service during active exams).
  - Facilitated **independent deployment, testing, and updates** of each service.
  - Used **Spring Cloud with Eureka** for service discovery and communication.

4. Deployment Setup

- Considered cloud platforms like AWS or GCP but opted for **on-premise LAN deployment** due to:
  - **Budget constraints** – no additional cloud cost for compute/storage.
  - **Convenience** – quick setup, restart, and monitoring during lab sessions.
  - **Data privacy** – student data remains within the institutional network.
- Deployed on **departmental lab machines** using LAN-based access.

- Used **Docker** for containerizing services and managing environments.

5. **Load Testing**

- Performed stress and concurrency testing using **Apache JMeter**.
- Test Scenarios:
  - Simulated increasing load: 100 → 200 → 500 → 700 → 1000 → 2000 users.
  - Measured key performance metrics:
    - **Response time under stress** – observed slight increase but remained within acceptable limits.
    - **CPU & Memory usage** – monitored to avoid overutilization.
    - **Disk I/O** – ensured database operations didn't become a bottleneck.
    - **System behavior under high concurrency** – no service crashes or major lag.
- Outcome:
  - Platform handled **up to 2000 concurrent users** with consistent performance.
  - Microservices structure allowed **efficient load distribution and fault isolation**.

4.1.2 Procedures Adopted

To ensure the successful development and deployment of the PrepPro Quiz Portal, a structured and iterative approach was adopted, combining elements of agile methodology, modular design principles, and best practices in software engineering. The following procedures outline the key stages and strategies that guided the development lifecycle of the project:

1. Requirement Gathering and Analysis:

- Identified pain points of traditional quiz systems through informal interviews with faculty and students.
- Documented:
  - ➔ Functional Requirements – quiz creation, analytics, user roles, quiz management.
  - ➔ Non-Functional Requirements – scalability, security, responsiveness, ease of deployment.

2. System Design and Architecture Planning:

- Adopted a microservices architecture for modularity and scalability.
- Services included:

    - User Service
    - Quiz Service
    - Question Service
    - Email Notification Service
    - QPF Entry Service
- Used REST APIs for communication and Eureka Server for service discovery.
- API Gateway was used as the unified entry point for all requests.

3. Technology Stack Selection:

- Backend: Java with Spring Boot – robust and scalable.
- Database: MongoDB – for flexibility and horizontal scaling.
- Caching & Messaging: Redis – improved performance and async handling.
- Frontend: React – dynamic, fast, and user-friendly.
- Containerization: Docker – for consistent deployment.

4. Module-Wise Development and Integration:

- Each microservice was developed independently, adhering to the Separation of Concerns principle.
- Performed unit and integration testing for each module.
- Integrated services using Feign Clients and tested communication over Docker bridge network.

5. Implementation of Security and Authentication

- Used JWT (JSON Web Token) for secure user authentication.
- Enforced Role-Based Access Control (RBAC) for authorized access.
- Implemented password encryption and email verification mechanisms.

6. UI Development and User Experience Optimization:

- Used JWT (JSON Web Token) for secure user authentication.
- Enforced Role-Based Access Control (RBAC) for authorized access.
- Implemented password encryption and email verification mechanisms.

7. Deployment, Testing, and Optimization:

- Deployed services using Docker on a VM running Ubuntu (LAN-based).

- Load testing was conducted using Apache JMeter with up to 2000 users.
- Optimizations included:
  - Redis caching
  - MongoDB query refactoring

8. Incorporating Recovery and Monitoring Mechanisms:

- Implemented auto-resume feature using browser cache (within 5 minutes).
- Integrated proctoring features like tab switch detection and session termination.
- Maintained logs and performance metrics for system health monitoring.

9. Feedback and Iterative Enhancement:

- Post-deployment feedback from users helped refine features.
- Improvements made to:
  - Usability (UI/UX)
  - System alerts
  - Report generation and admin modules.
- Ensured systems evolved with real-world academic use cases.

4.1.3 Techniques Developed

The development of the PrepPro Quiz Portal involved the creation and integration of several custom techniques and adaptations tailored to the specific requirements of academic assessment systems. These techniques ensured the system's reliability, usability, scalability, and adaptability to real-world exam scenarios. Below are the major techniques developed throughout the project:

1. Microservices-Based Modular Architecture:

A key technique adopted was the decomposition of the entire platform into logically independent microservices. Each service—such as user management, question handling, quiz scheduling, email notifications, and quiz profile tracking—was developed as a self-contained unit with its own database and communication contracts. This architecture allowed services to scale independently, improved fault tolerance, and made the system highly maintainable and future-ready for additional integrations.

2. RESTful Service Communication using Feign Clients:

To facilitate efficient inter-service communication, REST APIs were used across services, and Feign clients were implemented to abstract HTTP communication details. This ensured better readability, reusability of endpoints, and simplified integration between services like the User Service and Email Notification Service.

3. Stateless, Secure Authentication using JWT:

To secure the platform, a stateless authentication mechanism was implemented using JWT (JSON Web Tokens). Each user, upon logging in, received a signed token containing their role and identity claims. This token was then attached to each request, eliminating the need for server-side session storage and ensuring scalability. The system verifies the token for every incoming request, enforcing strict access control based on user roles.

4. Redis-Based Caching and Asynchronous Messaging:

Redis was used for two critical functionalities: caching and messaging. Frequently accessed data—such as user profiles or quiz metadata—was cached to reduce MongoDB access latency. Additionally, Redis Pub/Sub channels were used to decouple email notifications from core service logic, allowing asynchronous processing of events like user registration or password recovery without delaying API responses.

5. Tab-Switch Detection and Violation Tracking:

To simulate a semi-proctored environment, the system includes tab-switch detection during quizzes. JavaScript listeners on the frontend track tab switches, and a warning mechanism is triggered. Exceeding a defined threshold results in automatic test termination. This feature improves test integrity, especially for remote assessments.

6. Quiz Auto-Recovery and Resume Mechanism:

Understanding the possibility of unintentional test disruptions (like browser crashes or accidental refreshes), a quiz recovery mechanism was built using browser storage. Upon reconnection within a 5-minute window, the system retrieves the cached state and resumes the test from where it was left off. This ensures fairness and reliability, particularly in low-connectivity environments.

7. Bulk Uploads via CSV Parsing:

For administrative convenience, CSV-based upload features were implemented for users and question banks. Custom parsers were developed to validate data integrity, map content to appropriate models, and report upload errors in a user-friendly manner. This dramatically reduces the manual effort required for content entry.

8. Dynamic Quiz Timer Management:

PrepPro implements a robust timer mechanism at the quiz level to simulate real exam conditions. Each quiz is assigned a fixed duration, during which students can freely navigate between questions and submit answers. The timer ensures accurate time tracking and automatic quiz submission once the allotted duration elapses. The answer state is preserved in real-time to prevent data loss due to accidental tab closure or disconnection, ensuring a smooth and reliable test-taking experience.

9. Analytics and Progress Tracking Engine:

To support deep insights into student performance, a lightweight analytics engine was designed to track metrics like quiz scores, time spent per question, accuracy rate, and batch-wise performance trends. The results are visualized through the admin dashboard, helping educators personalize intervention strategies.

10. Containerized Infrastructure with Docker Networking:

All services were containerized using Docker and deployed in a custom bridge network. This approach ensured isolation, simplified networking, and reproducibility across environments. It also allowed for secure port exposure—where only the UI was publicly accessible—reducing the system's attack surface.

4.1.4 Methodologies

Microservices Architecture

- Application divided into independently deployable services:
    - User management
    - Quiz administration
    - Question handling
    - Email notifications
    - Service registry

- Each service:
  - Has its own data store
  - Communicates via RESTful APIs using Feign clients
- Benefits:
  - Better scalability and fault isolation
  - Easier debugging and maintenance
  - Clear separation of concerns

Agile Development

- Iterative development with weekly sprints
- Regular feedback sessions with stakeholders
- Benefits:
  - Features aligned with academic and user needs
  - Timely iterations and fast adaptation to changes
  - Continuous improvement and rapid delivery

DevOps Practices

- CI/CD pipelines implemented via GitHub Actions
- Automated steps for every code commit:
  - Code quality checks
  - Unit tests
  - Docker image creation
  - Deployment to staging
- Benefits:
  - Reliable and frequent deployments

  - Minimized integration issues
  - Smooth release cycles

Containerization

- All services containerized using Docker
- Custom Docker network for secure inter-service communication over private network
- Docker Compose for easy infrastructure setup and teardown
- Benefits:
  - Consistent environments across development, staging, and production
  - Faster developer onboarding
  - Simplified server deployments

Test-Driven Development (TDD)

- Writing tests before implementing functionality
- Tools used:
  - JUnit for unit testing
  - Mockito for mocking and validation
- Focus on:
  - Unit tests for individual components
  - Integration tests for service interactions
- Benefits:
  - Higher code quality and developer confidence
  - Early detection of edge cases and bugs

4.1.5 Algorithms Developed

The PrepPro Quiz Portal incorporates a range of intelligent and domain-specific algorithms to provide a secure, efficient, and user-friendly testing environment. These algorithms serve crucial functions such as quiz evaluation, randomization, caching, behavior monitoring during proctored assessments, bulk data handling, and session recovery. Below is a comprehensive overview of the key algorithms developed during the project lifecycle:

1. **Quiz Evaluation Algorithm**

   The quiz evaluation algorithm is responsible for assessing user responses post-submission. It handles different types of questions—such as single-correct MCQs, MSQs (multiple-select), and numericals. For MSQs, the algorithm implements partial marking by comparing the submitted set of answers with the correct answer set, using intersection and difference operations. The scoring logic supports customizable weightage, penalties for incorrect answers, and analytics on accuracy and time spent per question, which are crucial for performance analysis.

2. **Redis-Based Caching and Fast Access Algorithm**

   A customized caching algorithm is developed using Redis to accelerate data retrieval and reduce redundant DB hits. Frequently accessed data—like active quiz sessions, user tokens, and quiz metadata—is cached in Redis with expiration policies. This caching approach drastically improves response time for token verification and quiz start routines. The system is tuned to invalidate or update the cache upon user logout, quiz submission, or admin-triggered events.

3. **Proctored Assessment Warning System Algorithm**

   In proctored assessments, a behavioral monitoring algorithm tracks browser events such as tab switch, reloads, and console access. Each infraction increments a warning counter. Upon reaching a predefined threshold (e.g., three warnings), the algorithm invokes session termination procedures. All infractions are logged with timestamps, enabling audit trails for instructors. This helps enforce discipline during assessments without requiring third-party monitoring tools.

4. **Bulk Upload Deduplication and Validation Algorithm**

   The system includes an optimized parser that processes bulk-upload CSVs for user and question imports. The algorithm performs header validation, enforces data constraints, and checks for duplicate entries using hashing and database lookups. On detecting duplicates, it flags specific rows and continues processing the

rest, returning both success and error logs. This approach ensures robustness and resilience in high-volume admin operations.

5. **Session Recovery and Resumption Algorithm**

One of the standout features of PrepPro is its session recovery mechanism, which ensures minimal data loss during unexpected disruptions such as power cuts, browser crashes, or network failures. This is achieved through a hybrid **client-server algorithm** designed as follows:

- **Client-Side State Backup**: Every 30 seconds (configurable), the user's quiz state—current question index, timer state, selected answers—is stored in the browser's local storage.

- **Crash Detection and Grace Period**: If the browser unexpectedly closes or the user disconnects, a 5-minute grace period is initiated. Upon re-login, the system checks for a valid incomplete session in Redis or the browser's cache.

- **Resume Logic**: If a valid session exists, the quiz resumes exactly from the last checkpoint, including restored answers and time left. If the grace period expires or no session is found, the system flags the attempt as "abandoned."

This algorithm adds fault tolerance to the platform, allowing students to continue their exams with minimal interruption and reduces stress caused by unexpected technical failures.

4.2 Important Derivations and Formulae

While the PrepPro Quiz Portal is a software system at its core, several mathematical derivations and logical formulae are embedded within the platform to ensure accurate evaluation, insightful analytics, and efficient runtime behavior. This section outlines the key formulas used in the evaluation logic, performance tracking, recovery logic, and caching mechanisms.

1. Total Score Calculation (No Negative Marking)

The scoring engine assumes each question carries **1 mark** and **no penalty is applied** for incorrect responses. Therefore, the formula is simplified as:

$$Total\ Score\ =\ C$$

Where:

- C = Number of correctly answered questions (Single Correct)

2. Accuracy and Attempt Rate

To analyze how effectively a student is performing, the following metrics are used:

$$Accuracy\ (\%)\ =\ \left(\frac{Correctly\ Answered\ Questions}{Attempted\ Questions}\right) \times 100$$

$$Attempt\ Rate\ (\%)\ =\ \left(\frac{Attempted\ Questions}{Total\ Questions}\right) \times 100$$

These are displayed in dashboards to give students and faculty a quick understanding of test-taking behavior.

3. Proctoring Threshold Logic

To maintain exam integrity, the system tracks activities like tab switching and unauthorized navigation. The behavior is governed by:

$$W\ \geq\ T\ \Rightarrow \text{Terminate Test}$$

Where:

- W = Number of warnings issued (e.g., tab switches)
- T = Maximum allowed warnings before termination (e.g., 3).

Once the threshold is crossed, the platform immediately ends the quiz and flags the session.

4. Recovery Mechanism (Auto Resume with Cache)

In case of unintentional disconnection (tab close, crash), a recovery grace period is given:

$$T_{reconnect}\ \leq\ T_{grace}\ \Rightarrow\ \text{Resume Test from Cache}$$

Where:

- $T_{reconnect}$ = Time user takes to return to the quiz.
- T_{grace} = Maximum allowed reconnection time (5 minutes)

Cached answers are stored in the browser, allowing test resumption without loss of progress.

5. Session Timeout Logic

To automatically handle idle or disconnected sessions:

$$T_{idle} > T_{limit} \Rightarrow \text{Auto-logout}$$

Where:

- T_{idle} = Time since last action by user
- T_{limit} = Idle timeout threshold (configurable, e.g., 15 minutes)

These formulae and logic constructs provide the backbone for PrepPro's quiz evaluation, analytics, behavioral enforcement, and user session management. They ensure a fair, secure, and responsive test environment, supporting both student success and administrative control in a scalable, self-hosted ecosystem.

CHAPTER 5

Chapter 5

RESULTS & DISCUSSIONS

5.1 Outcomes

The **PrepPro Quiz Platform** has successfully met its primary objectives of providing a reliable, scalable, and user-friendly online assessment system tailored for placement preparation in academic environments. Key accomplishments of the platform are summarized below:

Functional & System-Level Outcomes

- **Efficient Role-Based Access:**
  - Multiple user roles—**Admin**, **Faculty**, and **Student**—with tailored access and UI/UX experience.
  - Ensures clear separation of responsibilities and secure data flow.
- **Simplified Quiz Creation & Management:**
  - Faculty can schedule quizzes with **random question generation**, **section-wise timers**, and **tag-based filtering**.
- **Automated Report Generation:**
  - Downloadable **Excel reports** with scores, section-wise analysis, and student-wise performance metrics.
- **Bulk Upload Support:**
  - CSV upload for **student registration** and **question bank creation**, improving admin efficiency.
- **High Concurrent User Handling:**
  - Optimized backend with **Dockerized microservices** efficiently handled **1000+ concurrent users**.
- **Secure Authentication and Authorization:**
  - **JWT-based login** with strict **role-based route access** for all users.
- **Real Exam Simulation Environment:**
  - **Full Screen mode**, **auto-submit on timeout**, and **distraction-free UI** mimicking actual online exams.
- **Easy Deployment & Maintenance:**
  - Deployment via **Docker + Docker Compose** for consistent setup.
  - Native support for **Ubuntu systemctl** for service control.

```
root@Koustubh: /mnt/d/New
root@Koustubh:/mnt/d/New folder (2)/QUIZ-PORTAL# systemctl status quizapp.service
○ quizapp.service - Quiz App Container Management Service
     Loaded: loaded (/etc/systemd/system/quizapp.service; disabled; preset: enabled)
     Active: inactive (dead)
root@Koustubh:/mnt/d/New folder (2)/QUIZ-PORTAL# systemctl start quizapp.service
root@Koustubh:/mnt/d/New folder (2)/QUIZ-PORTAL# systemctl enable quizapp.service
Created symlink /etc/systemd/system/multi-user.target.wants/quizapp.service → /etc/systemd/system/quizapp.service.
root@Koustubh:/mnt/d/New folder (2)/QUIZ-PORTAL# systemctl status quizapp.service
● quizapp.service - Quiz App Container Management Service
     Loaded: loaded (/etc/systemd/system/quizapp.service; enabled; preset: enabled)
     Active: active (exited) since Wed 2025-04-23 08:25:50 UTC; 25s ago
   Main PID: 1031 (code=exited, status=0/SUCCESS)

Apr 23 08:25:53 Koustubh quiz-service.sh[1424]: quiz
Apr 23 08:25:53 Koustubh quiz-service.sh[1031]: 🔄 Starting existing container: serviceregistry
Apr 23 08:25:59 Koustubh quiz-service.sh[1539]: serviceregistry
Apr 23 08:25:59 Koustubh quiz-service.sh[1031]: 🔄 Starting existing container: apigateway
Apr 23 08:25:55 Koustubh quiz-service.sh[1649]: apigateway
Apr 23 08:25:55 Koustubh quiz-service.sh[1031]: 🔄 Starting existing container: user
Apr 23 08:25:56 Koustubh quiz-service.sh[1780]: user
Apr 23 08:25:56 Koustubh quiz-service.sh[1031]: 🔄 Starting existing container: emailservice
Apr 23 08:25:57 Koustubh quiz-service.sh[1895]: emailservice
Apr 23 08:25:57 Koustubh quiz-service.sh[1031]: ✅ All services started.
root@Koustubh:/mnt/d/New folder (2)/QUIZ-PORTAL# systemctl stop quizapp.service
root@Koustubh:/mnt/d/New folder (2)/QUIZ-PORTAL# systemctl status quizapp.service
○ quizapp.service - Quiz App Container Management Service
     Loaded: loaded (/etc/systemd/system/quizapp.service; enabled; preset: enabled)
     Active: inactive (dead) since Wed 2025-04-23 08:27:02 UTC; 10min ago
    Duration: 37.492s
     Process: 2373 ExecStop=/usr/local/bin/quiz-service.sh stop (code=exited, status=0/SUCCESS)
   Main PID: 1031 (code=exited, status=0/SUCCESS)

Apr 23 08:26:31 Koustubh quiz-service.sh[2564]: serviceregistry
Apr 23 08:26:31 Koustubh quiz-service.sh[2373]: 🔻 Stopping apigateway
Apr 23 08:26:41 Koustubh quiz-service.sh[2620]: apigateway
Apr 23 08:26:42 Koustubh quiz-service.sh[2373]: 🔻 Stopping user
Apr 23 08:26:52 Koustubh quiz-service.sh[2709]: user
Apr 23 08:26:52 Koustubh quiz-service.sh[2373]: 🔻 Stopping emailservice
Apr 23 08:27:02 Koustubh quiz-service.sh[2777]: emailservice
Apr 23 08:27:02 Koustubh quiz-service.sh[2373]: ✅ All containers stopped.
Apr 23 08:27:02 Koustubh systemd[1]: quizapp.service: Deactivated successfully.
Apr 23 08:27:02 Koustubh systemd[1]: Stopped quizapp.service - Quiz App Container Management Service.
root@Koustubh:/mnt/d/New folder (2)/QUIZ-PORTAL#
```

Fig. 5.1 *Active Docker Containers*

- **LAN Compatibility for Offline Exams:**
  - Can be hosted on **local servers** without the need for internet access.
- **Automation Scripts for Developer Productivity**
  - Wrote shell and batch scripts to:
    - Launch all containers and services with a single command
    - Start individual services (e.g., only backend or Redis)
    - Tear down and restart environments smoothly

```
root@Koustubh: /mnt/d/New
root@Koustubh:/mnt/d/New folder (2)/QUIZ-PORTAL/scripts# ls
automate.sh         check.sh         deploy-all.sh         quiz-service.sh   stopOnHost.sh
automate2.sh        checkAPIs.sh     installtion.sh        remove.sh         testOtheruser.sh
automateOnHost.sh   create-jars.sh   installtionUbuntu.sh  service.sh        uninstallation.sh
root@Koustubh:/mnt/d/New folder (2)/QUIZ-PORTAL/scripts#
```

Fig. 5.2 *Automation Scripts*

Development Practice & Industry-Standard Outcomes

- **End-to-End Product Engineering:**
  - Entire product development followed a **modular and scalable approach**, with team members simultaneously working on features like authentication, quiz APIs, admin analytics, and reporting.
- **GitHub Project Integration:**
  - A dedicated **GitHub repository** was created for version control.
  - All contributors were added as **collaborators** with proper access levels.

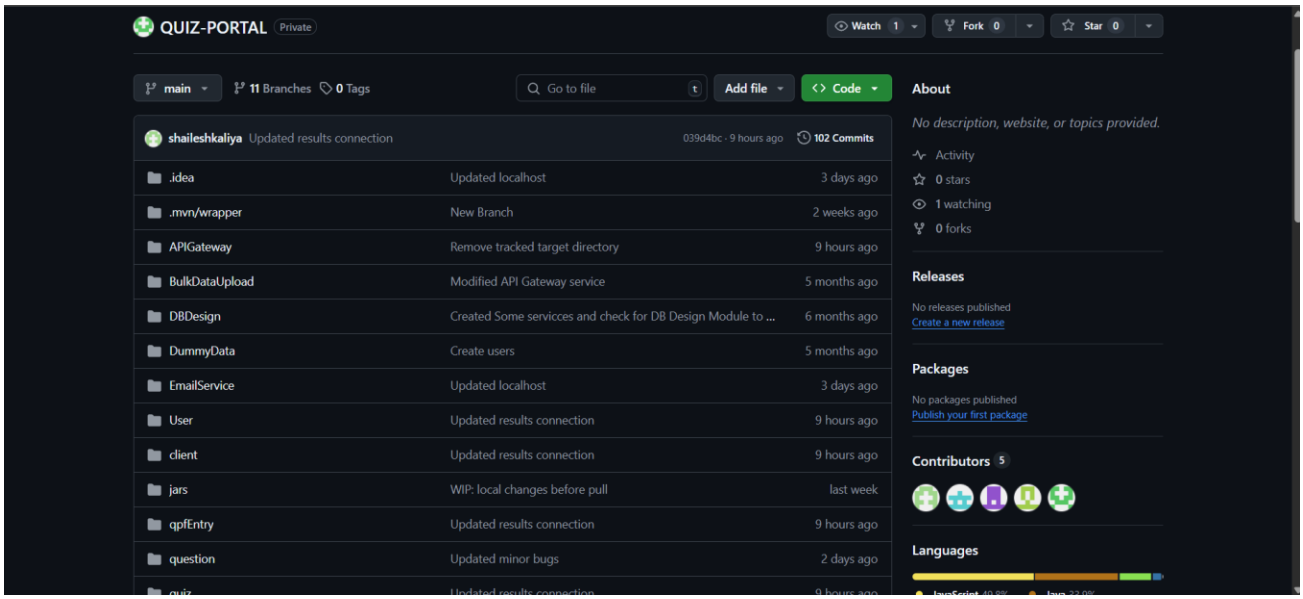


Fig. 5.3 *GitHub Repository for Project*

- **Industry-Level Git Practices Followed:**
  - Feature branches were created for independent modules.
  - Pull Requests (PRs) included **code reviews**, **rebasing**, and **conflict resolution** before merging.
  - Maintained:
    - **main** branch (stable, production-ready)
    - **dev** branch (in-progress but stable build)
    - **Feature** branches (individual modules in development)
    - Backup repositories for risk mitigation and recovery

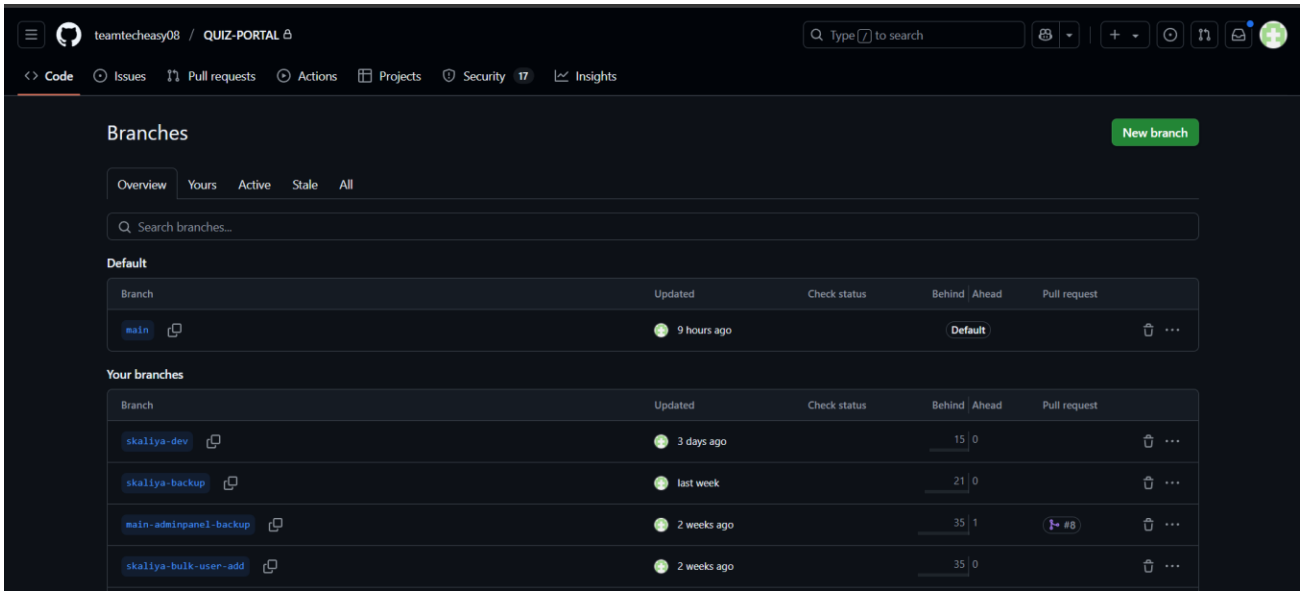


Fig. 5.4 main and dev *Branches*

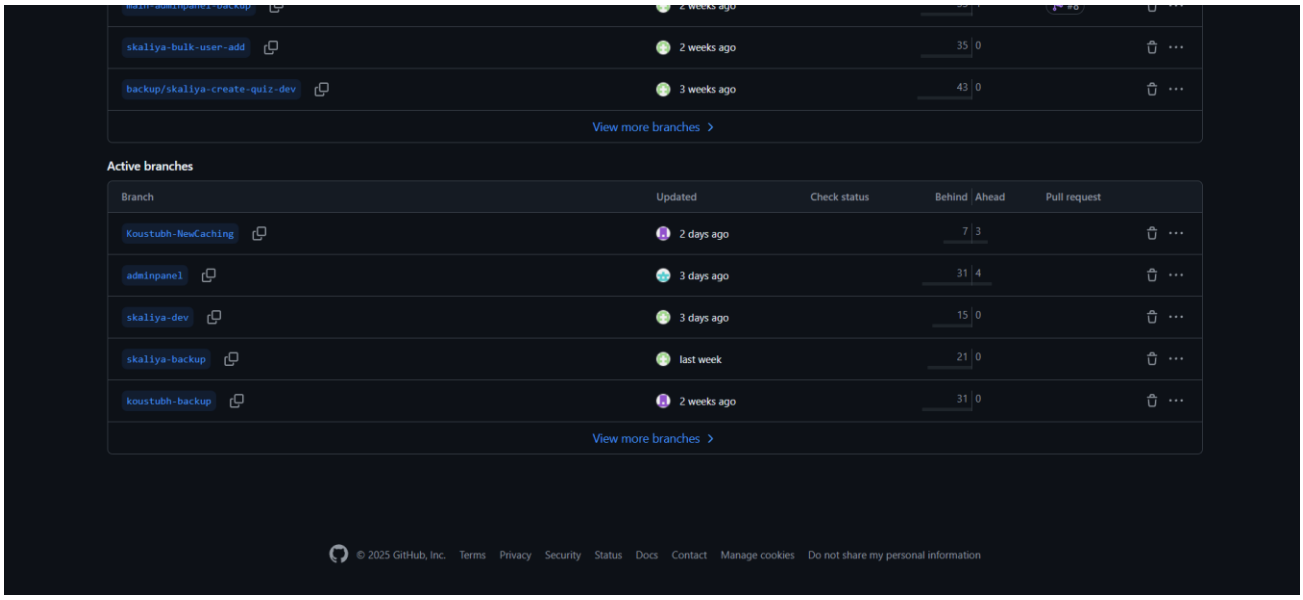


Fig. 5.5 *Feature Branches*

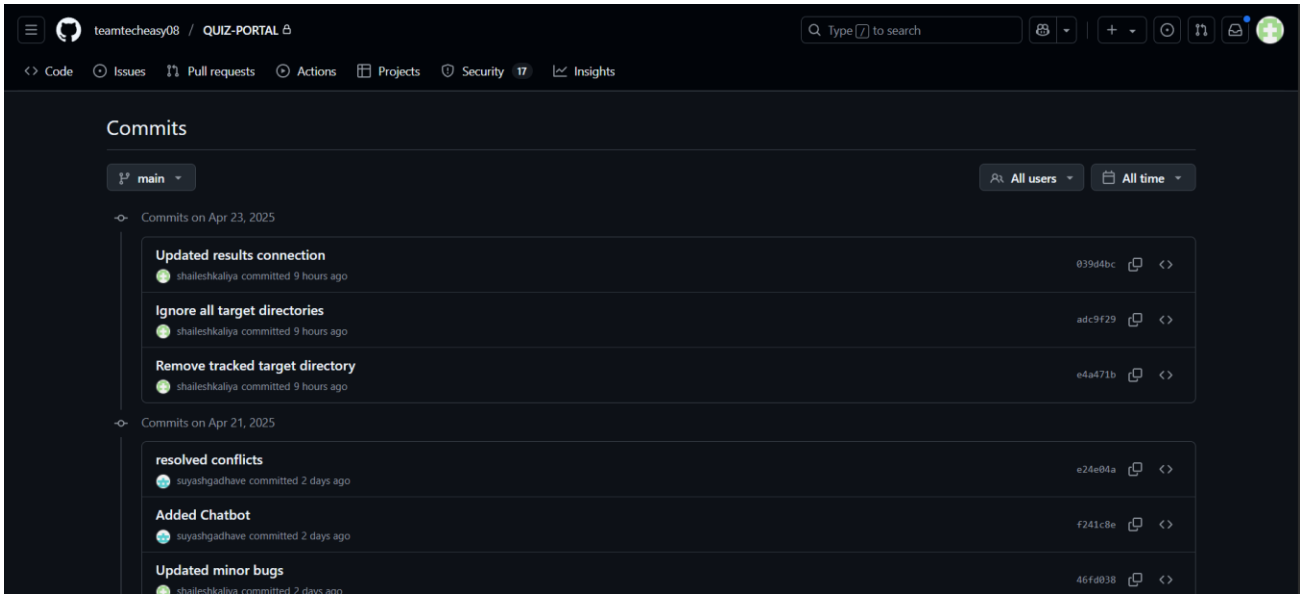


Fig. 5.6 *Commits and Merges*

- **Directory Structure Optimization:**
  - Followed **standard MVC architecture** across services.
  - Each microservice contained distinct folders for **controller**<Rest Layer>, **service**<interface>, **serviceimpl**<business logic>, **repository**, **config**, and **dto**<info, params, model objects>.

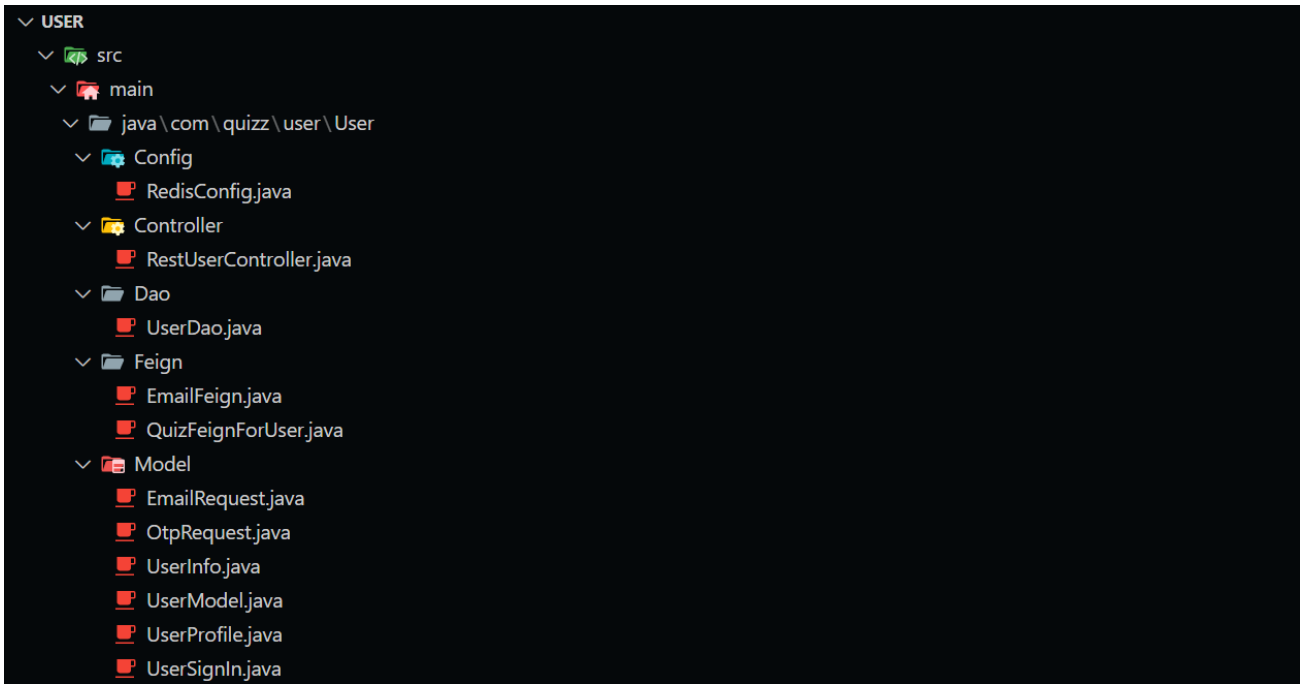


Fig. 5.7 *Backend Microservice Directory Structure*

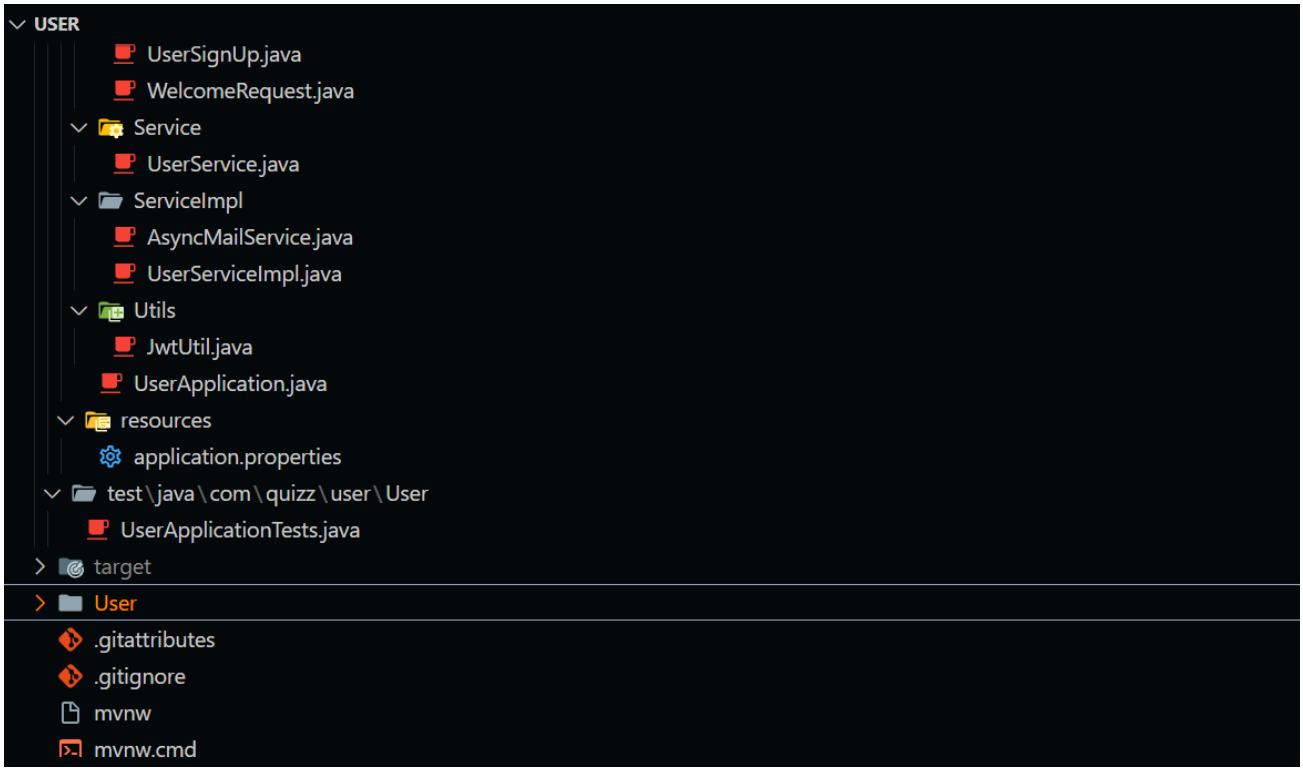


Fig. 5.8 *Backend Microservice Directory Structure*

- **Collaborative Workflow:**
  - All development and fixes were merged after thorough **manual testing** and **functional validations**.

5.2 Result analysis and Validations

Multiple tests were conducted to validate platform performance, usability, and system robustness:

Functional Testing

- All major features—quiz scheduling, question uploading, student login, and scoring—were rigorously tested across roles.
- The system handled complex quizzes with section-based timers and randomized question orders without error.

Load Testing

- Simulated user tests were conducted to evaluate concurrency support.
- The system maintained stable response times (<2 seconds) with 1000 simulated users logging in and attempting quizzes simultaneously.

Security Testing

- JWT authentication tokens were verified to ensure expiry and role validation.
- Role-based access control was validated for route protection (e.g., students cannot access admin routes).
- Basic penetration tests for SQL Injection, Cross-Site Scripting (XSS), and broken authentication were conducted and passed.

Performance Reports & Feedback

- Generated Excel reports for test analysis were found to be accurate and matched manual evaluations.
- Faculty reported satisfaction with automated insights such as score distributions, weak topic identification, and per-section analytics.

Deployment & Maintenance

- Docker-based deployment was successful on both Windows and Ubuntu systems.
- Admins could start/stop backend services via systemctl without technical complexity.

Test Results Summary Table

Table 5.1 *Test Cases Summary Table*

Category	Test Case / Scenario	Tool / Method	Expected Outcome	Status
Functional Testing	Quiz scheduling and timing validation	Manual + Postman API testing	Quiz scheduled with correct date, timer initiates	Passed

	Student login and attempt flow	UI flow test	Students can log in, view assigned quizzes	Passed
	Section-based timer and randomized questions	Internal logic checks	Timers isolated per section, question order randomized	Passed
Load Testing	1000 concurrent users attempting quiz	Apache JMeter	Avg response time < 2 sec, no crash	Passed
	Gradual load increase (100 → 2000 users)	Apache JMeter	System scales with minor latency increase	Passed
Security Testing	JWT expiry and role verification	Postman + Console Logs	Invalid/expired token = access denied	Passed
	Unauthorized route protection (Student → Admin route)	Route testing in browser	403 Forbidden error shown	Passed
	SQL Injection, XSS, Broken Authentication	Manual Pen Test	No vulnerabilities found	Passed

Performance Reports	Accuracy of auto-generated Excel results	Manual vs. Generated Report	Scores and feedback match manual evaluation	Passed
	Admin analytics (score dist., weak areas)	Visual + Data Inspection	Correct insights and visualizations	Passed
Deployment & Maintenance	Docker-based deployment (LAN/VM - Windows & Ubuntu)	Docker CLI + System Logs	Services up and running, isolated containers	Passed
	Admin controls via `systemctl`	Ubuntu server CLI	Start/stop without Docker/technical knowledge	Passed

5.3 Screenshots

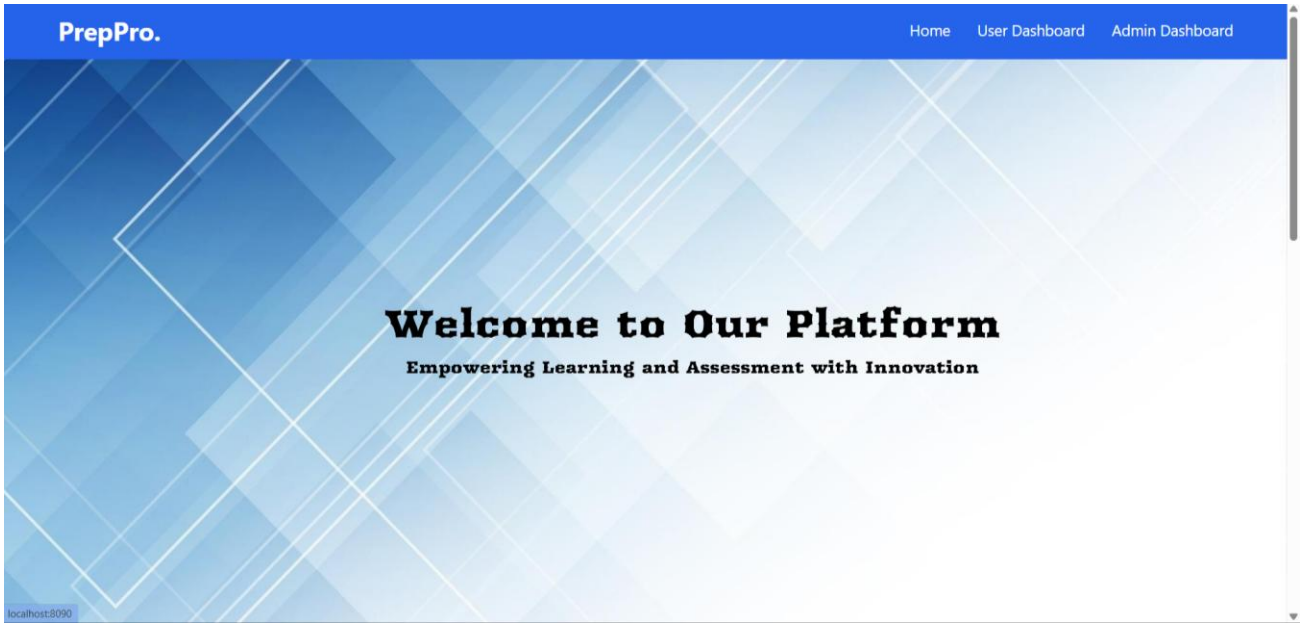


Fig. 5.9 *Home Page*

The landing page of the PrepPro portal, offering login options for both Admin and Users, serving as the primary access point to their respective dashboards.

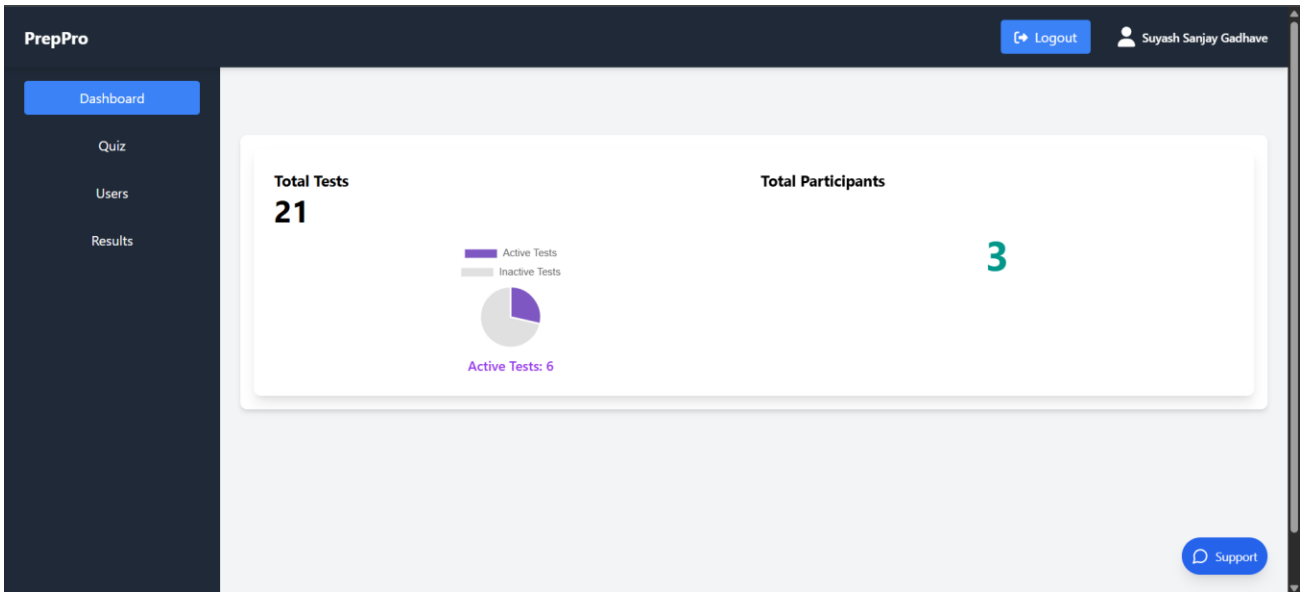


Fig. 5.10 *Admin Dashboard*

A centralized interface for administrators to view platform-wide metrics, such as the total number of quizzes created and participants enrolled.

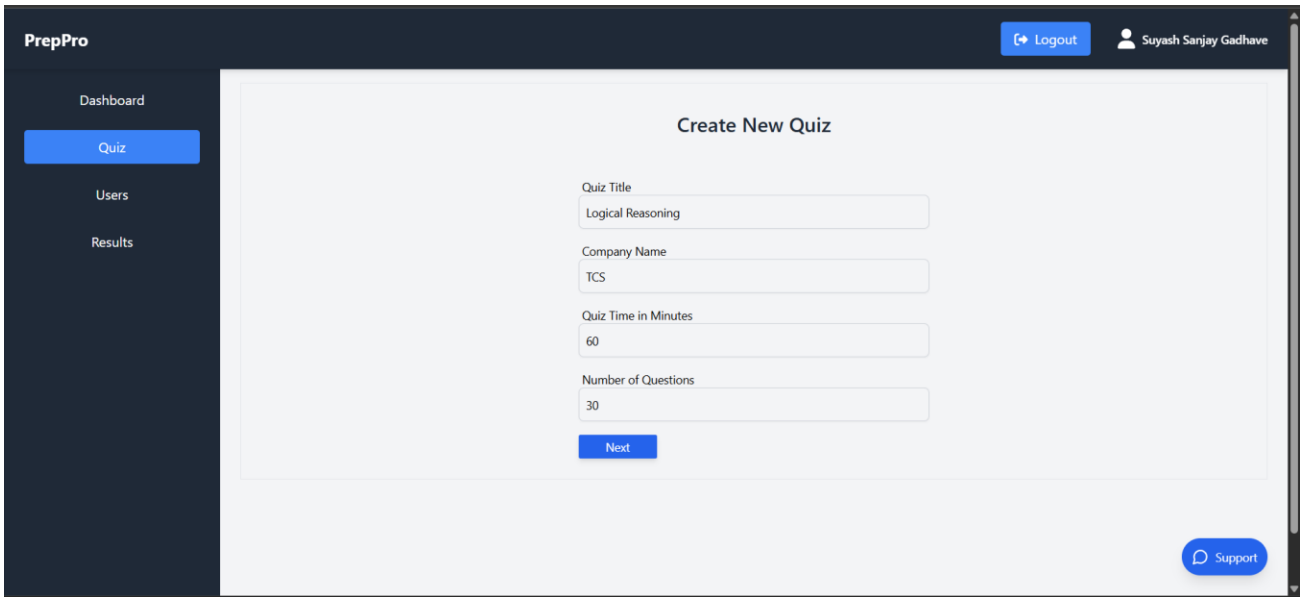


Fig. 5.11 *Create Quiz for Admin*

Allows admins to initiate a new quiz by defining its metadata, configuration settings, and other essential quiz-related properties.

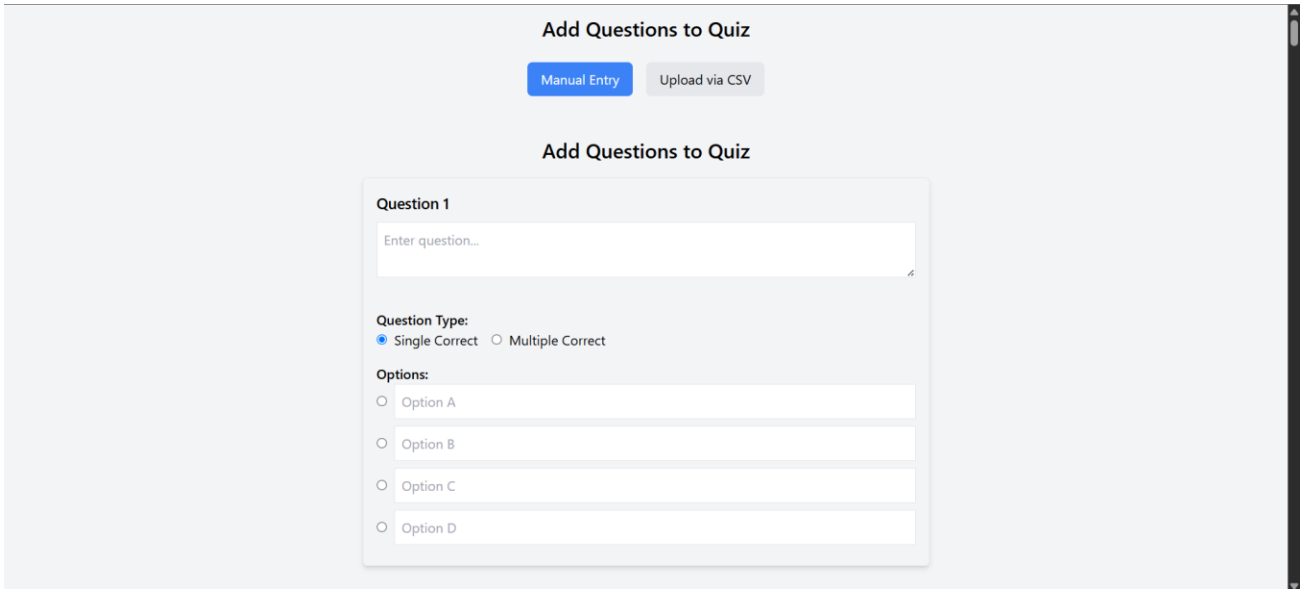


Fig. 5.12 *Add Question to Quiz*

Enables manual entry of individual questions into a selected quiz, with fields for question text, options, and correct answers.

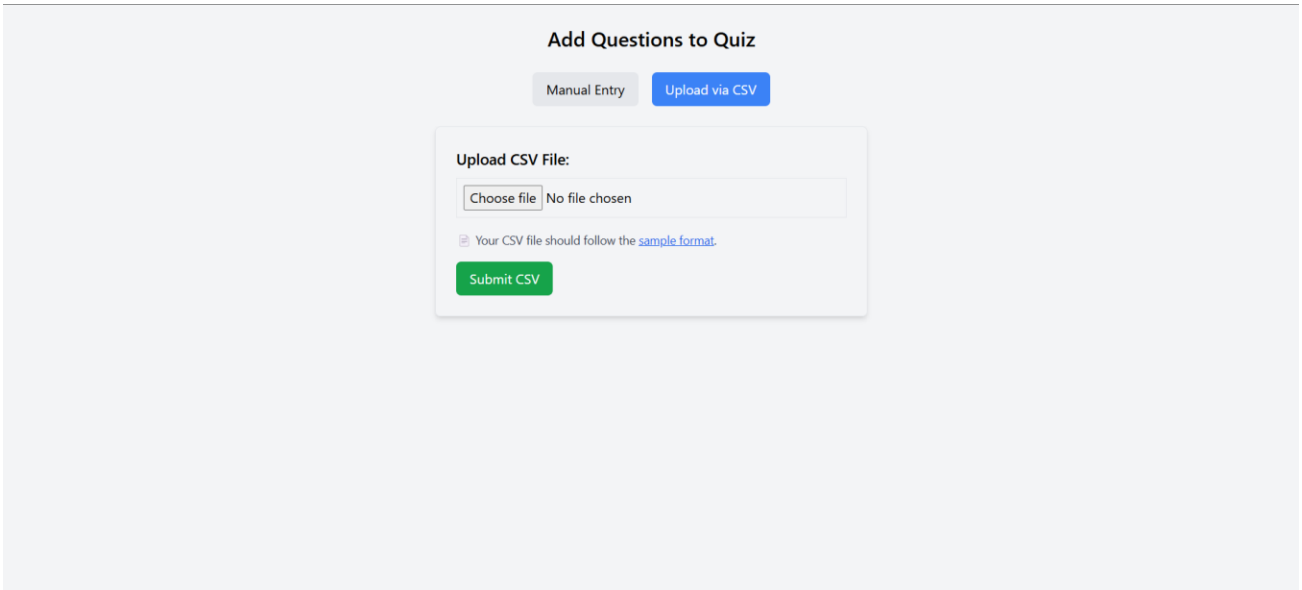


Fig. 5.13 *Add Question to Quiz using CSV*

Offers a bulk upload option where admins can import questions by uploading a properly formatted CSV file, streamlining the quiz creation process.

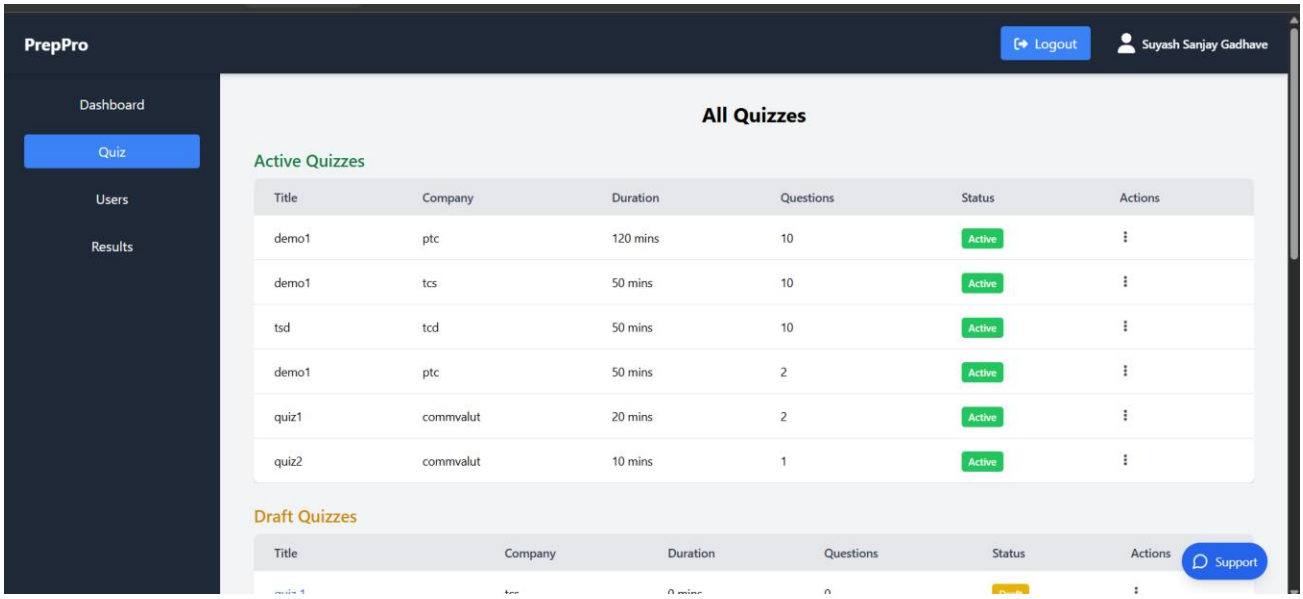


Fig. 5.14 *Show all Quiz to Admin*

Offers a bulk upload option where admins can import questions by uploading a properly formatted CSV file, streamlining the quiz creation process.

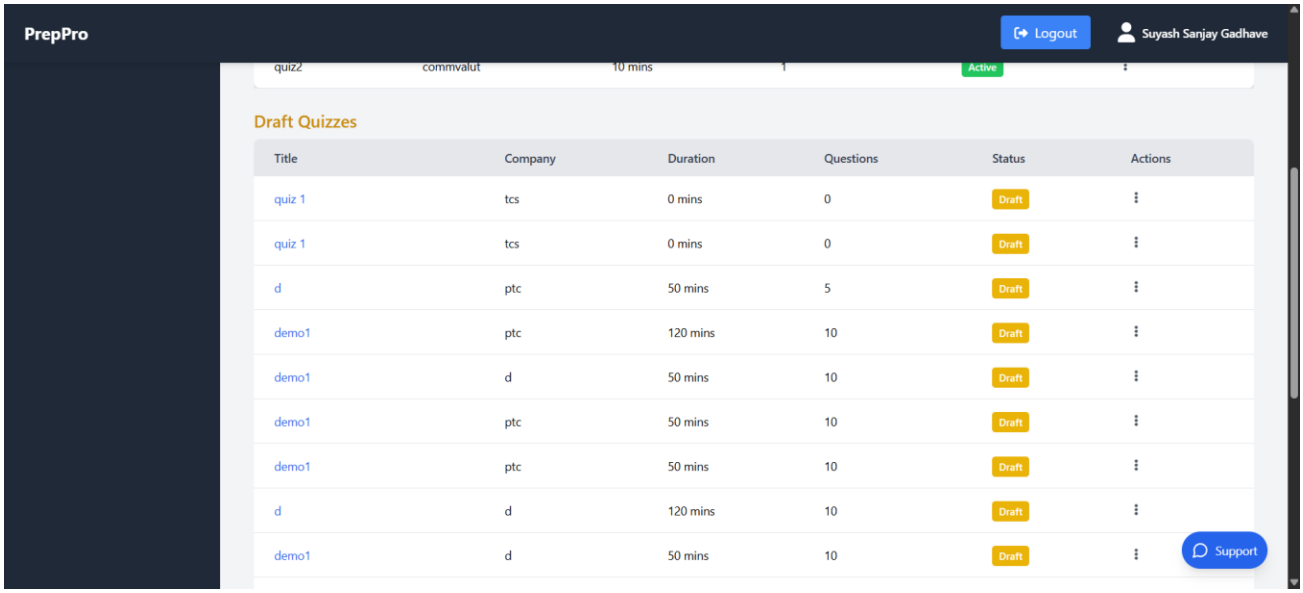


Fig. 5.14 *Show quizzes in draft mode*

Allows admins to access and manage quizzes that were saved as drafts and not yet published, supporting continued editing.

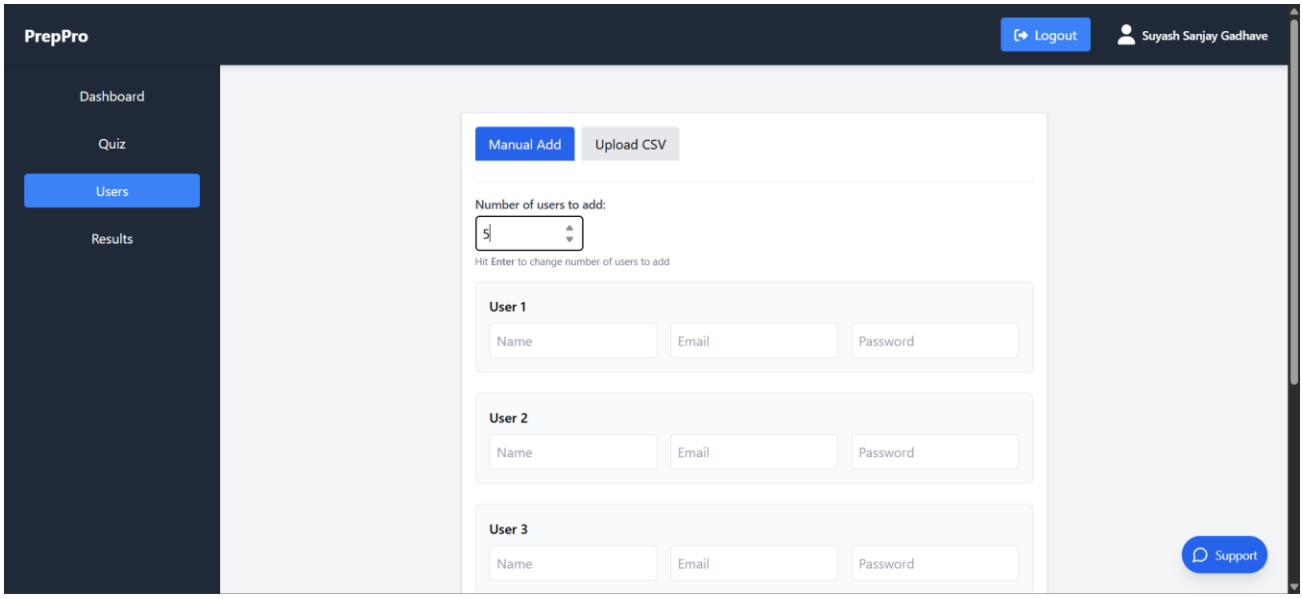


Fig. 5.15 *Add User Manually*

Facilitates the manual addition of individual users (students) by the admin, including basic details and credentials setup.

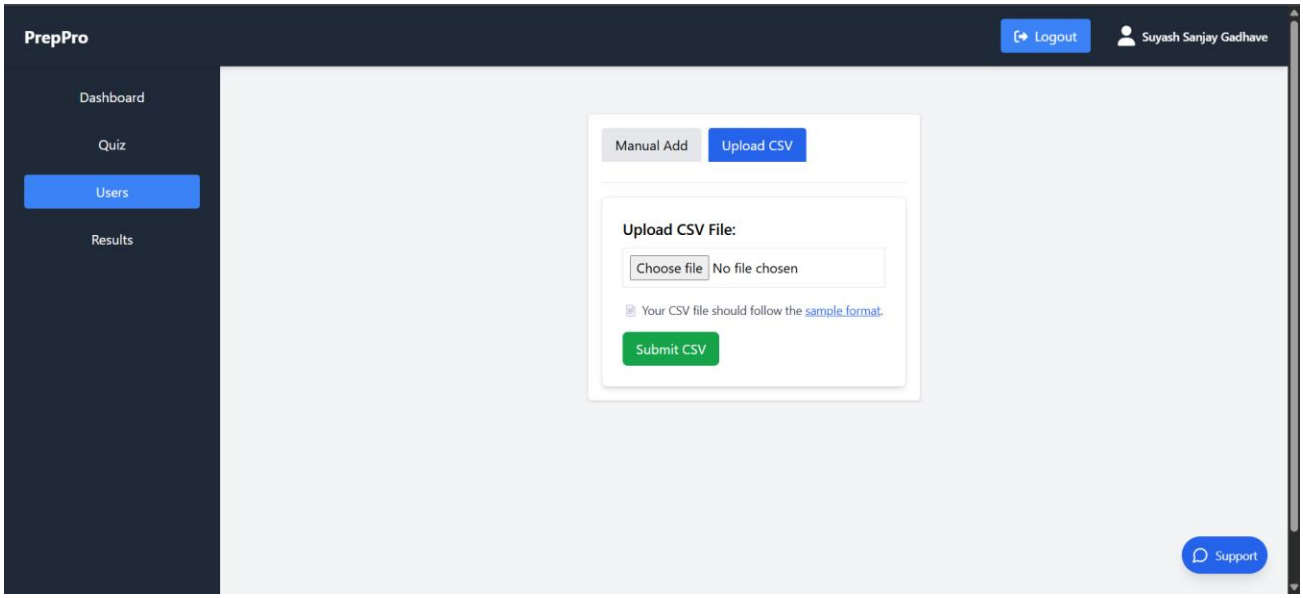
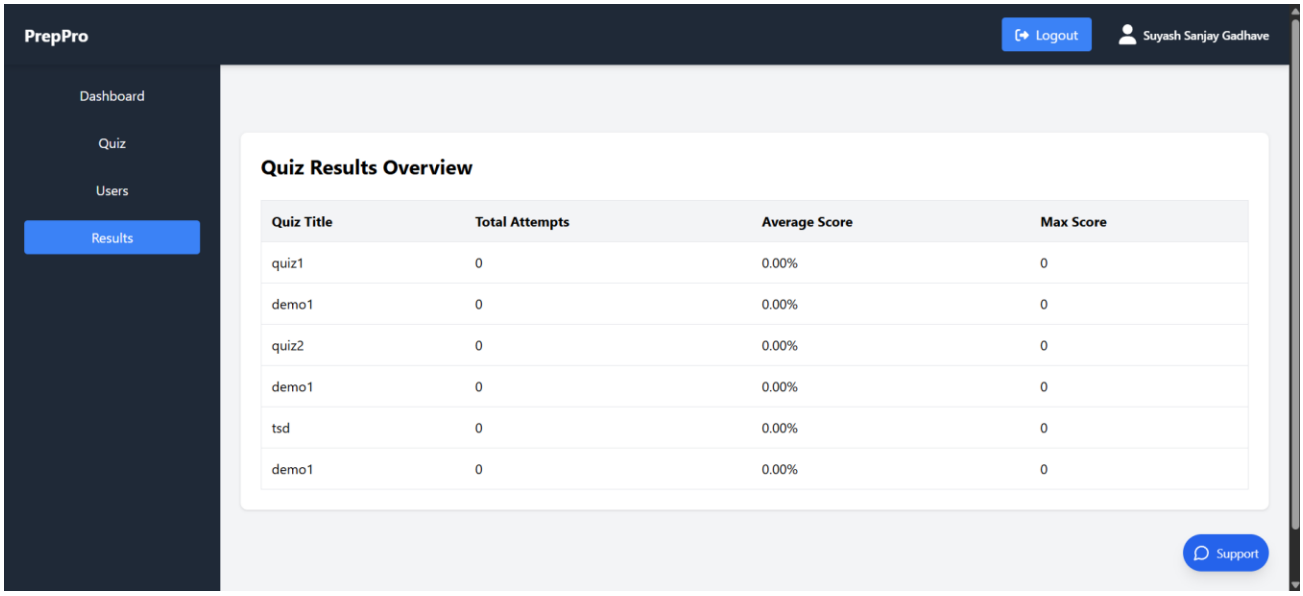


Fig. 5.16 *Add User using CSV*

Enables bulk user onboarding through CSV upload, helping admins efficiently register multiple users at once.



Quiz Title	Total Attempts	Average Score	Max Score
quiz1	0	0.00%	0
demo1	0	0.00%	0
quiz2	0	0.00%	0
demo1	0	0.00%	0
tsd	0	0.00%	0
demo1	0	0.00%	0

Fig. 5.17 *Quiz result panel for Admin*

Provides detailed performance results of students for each quiz, with options to export the data in multiple file formats.

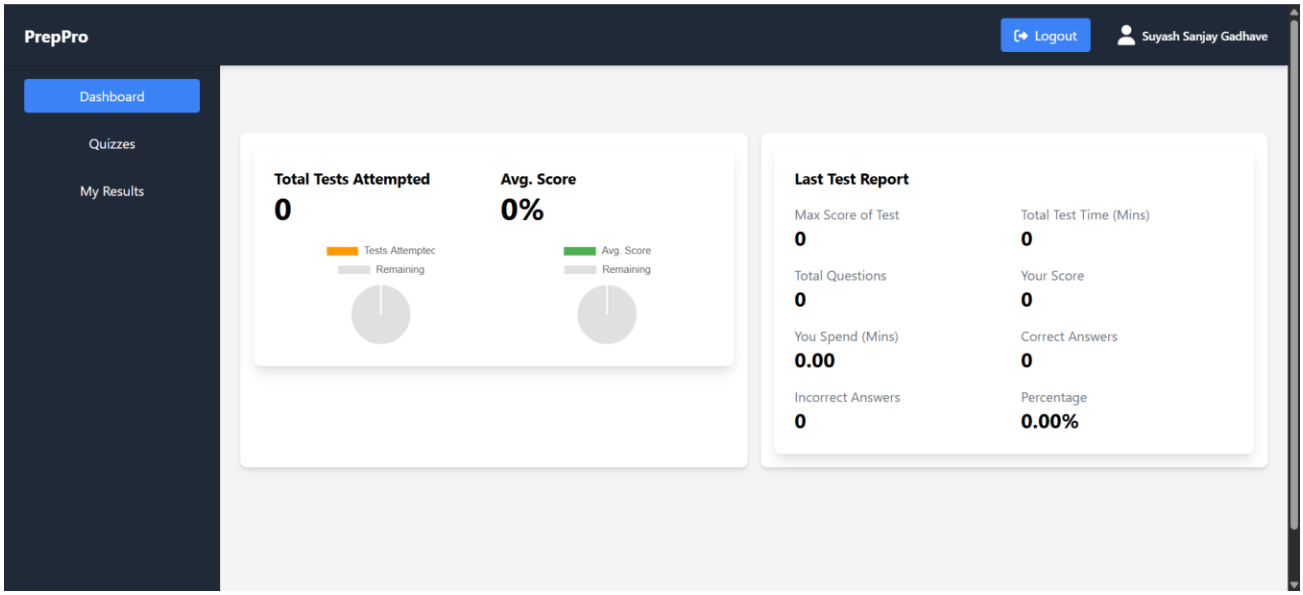


Fig. 5.18 *Dashboard for User*

A personalized dashboard for students displaying key performance metrics such as total attempts, average scores, and a summary of the latest test.

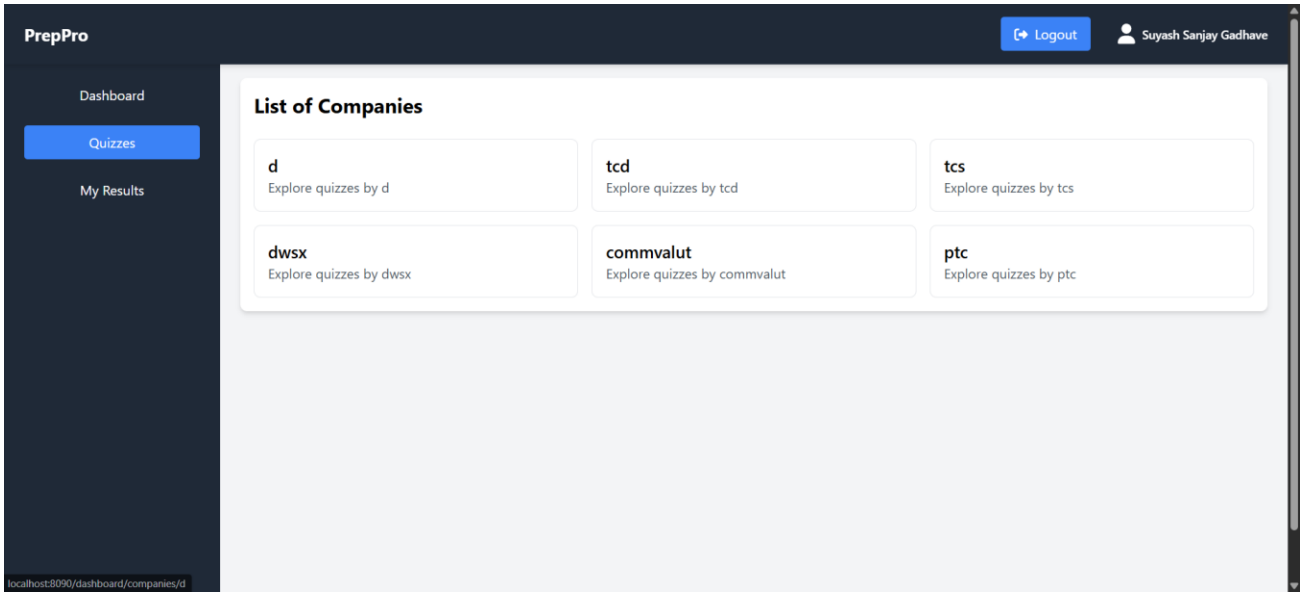


Fig. 5.19 *Quizzes section for User*

Organizes available quizzes by subject or company tags, helping users quickly find relevant assessments.

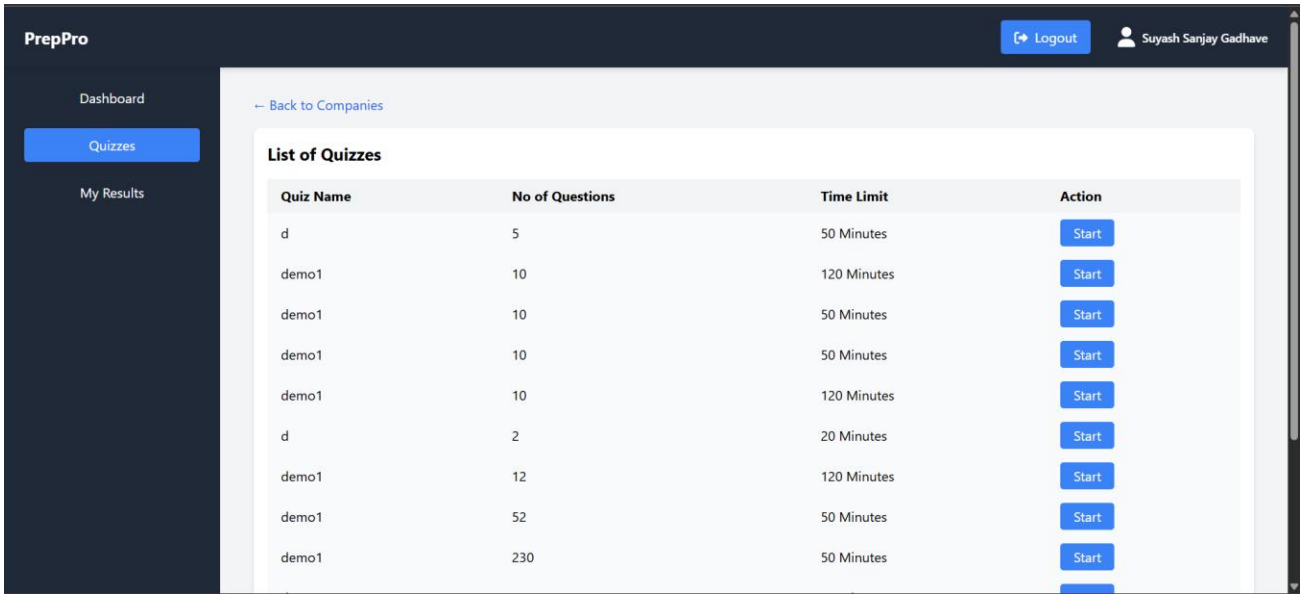


Fig. 5.20 *List of Quizzes as per Company*

An expanded view displaying all quizzes associated with a particular company or domain, aiding targeted preparation.

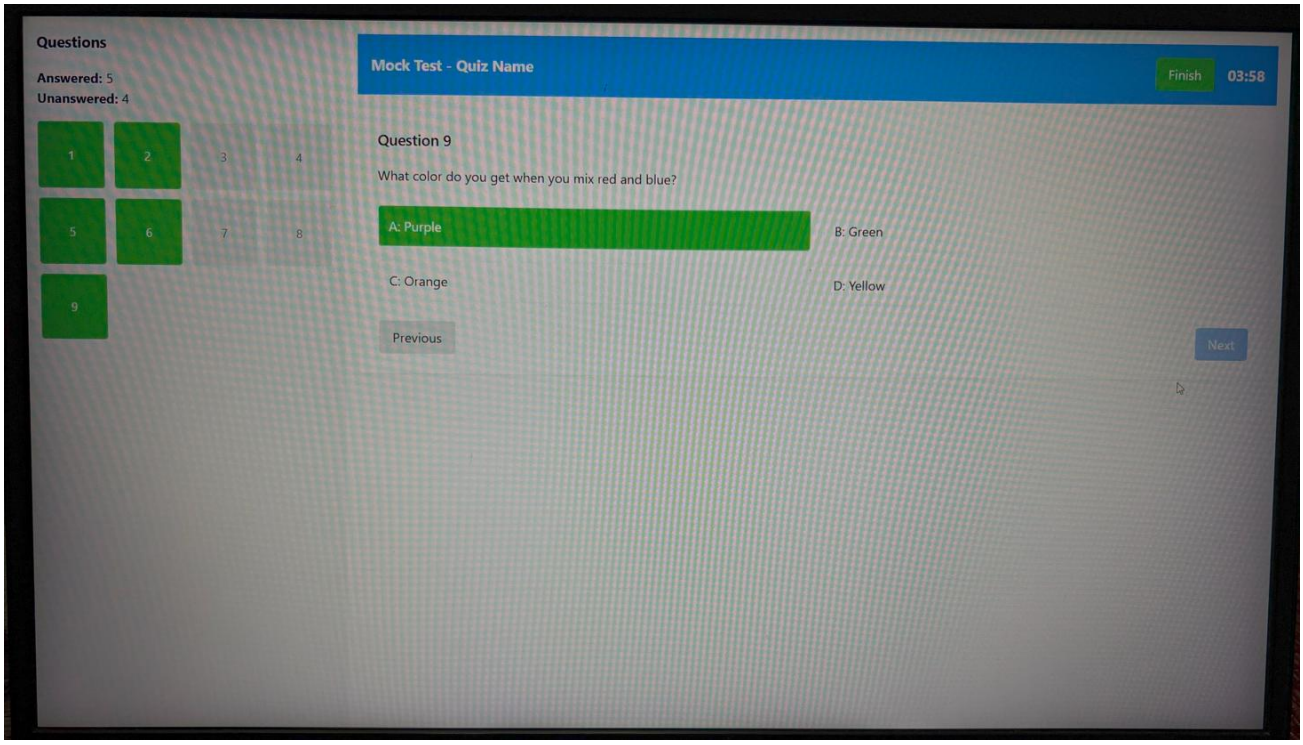


Fig. 5.21 *Quiz Portal*

The live quiz interface features timers, question navigation, and indicators for attempted and unattempted questions.

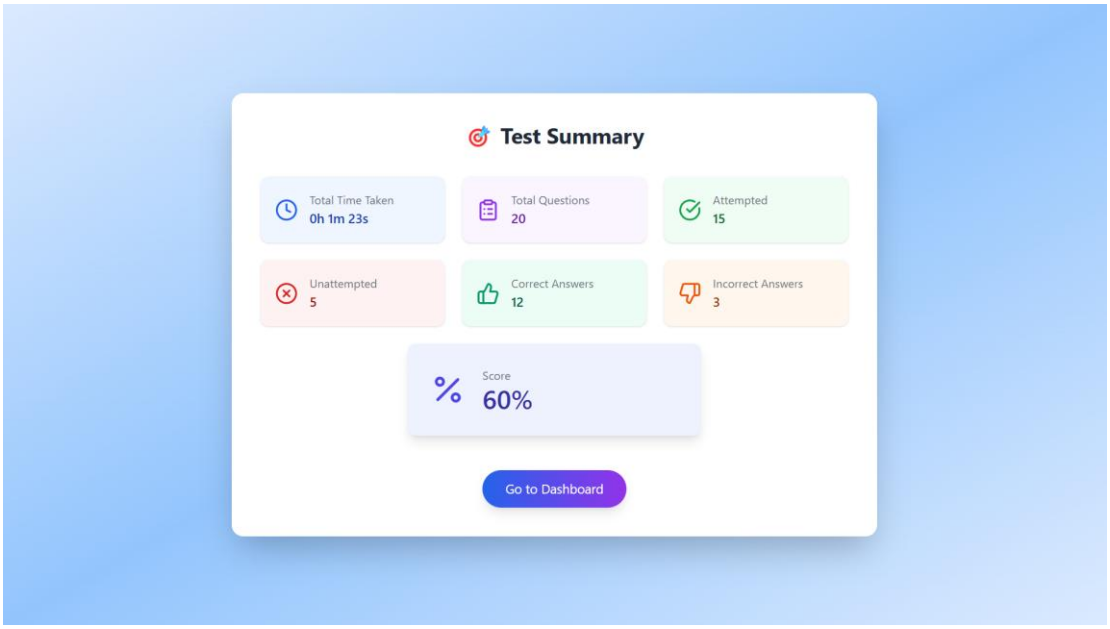


Fig. 5.22 *Result Page after submission*

Displays the result summary post quiz submission, including score breakdowns and performance analytics for the user.

CHAPTER 6

Chapter 6

CONCLUSION

6.1 Conclusion

The **PrepPro Quiz Platform** is a robust, scalable, and future-ready solution tailored for academic institutions to conduct seamless and high-quality assessments. It overcomes limitations of traditional systems by leveraging a **microservices-based architecture**, ensuring:

- **Modularity** and **independent service deployment**
- **Scalability** for increasing institutional demands
- **Flexible deployment** across cloud and on-premise setups

Key Highlights:

- **User-Friendly Interface:**
  - Intuitive design for both students and faculty
  - Faculty can create quizzes with:
    - Section-wise timers
    - Randomized question sequences
    - Customizable difficulty levels
- **Performance & Analytics:**
  - Real-time performance insights
  - Group and individual performance tracking
  - CSV-based bulk uploads for onboarding and question management
- **Realistic Exam Environment:**
  - Full-screen distraction-free mode
  - Responsive UI for all screen sizes
  - REST-based communication (lightweight, portable, easy to debug)
- **Security & Authentication:**
  - JWT-based secure login
  - Role-based access control
  - Secure, validated API interactions
- **Proctoring & Recovery Mechanisms:**

  - Tracks tab switches and suspicious activity
  - Auto-warnings and auto-submission after violations
  - Auto-recovery if the browser crashes or accidentally closes (restores session within 5 minutes)
- **Containerized Deployment:**
  - Docker-based setup for high concurrency and easy scalability
  - Supports deployment in cloud or offline campus networks

In summary, **PrepPro** combines usability, scalability, and security to offer a smart, institution-grade assessment system. With its focus on real-time feedback, system resilience, and realistic test simulations, it's an ideal upgrade from outdated systems—empowering educators and students alike.

In terms of performance, the system is designed to handle up to **2000 concurrent users** effectively, maintaining an average **response time of under 1000 ms** per request. Through horizontal scaling, load balancing, and efficient caching mechanisms, the system can ensure stability and low latency, even during peak traffic periods. These performance metrics ensure that the platform remains responsive and reliable, even as institutional requirements grow, making it an ideal upgrade from traditional systems.

6.2 Future Scope

The platform can be further enhanced by introducing the following features:

- **Mobile App Integration**
  Develop Android/iOS apps to improve accessibility and deliver push notifications for quiz updates and results.

- **LMS & ERP Integration**
  Seamless integration with existing institutional Learning Management Systems (LMS) or ERP tools to sync student records and exam schedules.

- **Multilingual Interface**
  Provide support for regional languages to cater to a broader audience in diverse academic institutions.

- **Proctoring Mode (Optional)**
  Integrate a lightweight monitoring system using webcam screenshots or behavior logging to mimic remote proctoring.

- **Custom Certificates and Feedback Reports**
  Enable downloadable certificates or skill analysis reports based on performance metrics.

- **API Exposure**

Offer external APIs for institutions to automate tasks like student onboarding, quiz uploads, or result retrieval.

6.3 Applications

The PrepPro platform can be deployed in various educational and institutional contexts:

- **Academic Institutions**
  For conducting internal tests, placement mock exams, and curriculum-based assessments.

- **Training & Coaching Centers**
  Ideal for test series, timed practice modules, and exam simulations for competitive exams.

- **Skill-Based Assessments**
  Useful for conducting certification exams or technical assessments based on predefined skill sets.

- **Corporate Training & Hiring**
  Companies can adapt platform for employee assessments, onboarding tests, or internship selection.

- **Government and Public Sector Exams (Optional Deployment)**
  Can be tailored to host mock tests for public service aspirants in a controlled LAN environment.

References

1. Ameen, A. A., & Al-Mashhadani, S. A. (2021). Design and Implementation of Online Examination System based on MSVS and SQL for University Students in Iraq. Scientific.net

2. Shourie, P., Anand, V., Chauhan, R., & Gupta, S. (2023). AI-based Online Proctoring System with YOLO-v3 & MMOD CNN. ResearchGate

3. Makkar, D. (2021). Create NodeJS Server for a Real-Time Quiz App. Medium

4. AKCoding. (2022). Microservices Architecture with Spring Boot & Kubernetes. Medium

5. Plotly. (2020). Automate Excel Reports with Python. Plotly

Appendix

1. Appendix A: Data sheets

1.1 Lines of Code by Language

This quiz portal is a full-stack microservice-based platform built using a wide variety of technologies. The application includes a React-based frontend, multiple Spring Boot microservices, deployment scripts, and utility tools.

Table: *Lines of Code by Language*

Language	Lines of Code	Description
Java	2,225	Core backend services (Spring Boot microservices)
CSS	1,945	Styling for frontend user interface
HTML	26	Basic templates and fallback content
XML	814	Spring configs, Maven POM files
Properties	430	Application configurations, message files
JavaScript	927	Includes frontend logic
JSX	6,808	React component views
Python	451	Utility scripts (one-time tools)
Shell Script	1,168	Deployment and automation scripts

```
root@Koustubh:~/CopyBack/QUIZ-PORTAL-main# for lang in java css html xml properties js jsx py sh;
> do
> echo "For Language $lang Line of Code is $(find . -name "*.$lang" -exec cat {} + | wc -l )"
> done;
For Language java Line of Code is 2225
For Language css Line of Code is 1945
For Language html Line of Code is 26
For Language xml Line of Code is 814
For Language properties Line of Code is 430
cat: ./client/node_modules/chart.js: Is a directory
cat: ./client/node_modules/fraction.js: Is a directory
For Language js Line of Code is 1250967
For Language jsx Line of Code is 6808
For Language py Line of Code is 451
For Language sh Line of Code is 1168
root@Koustubh:~/CopyBack/QUIZ-PORTAL-main#
```

Fig: *Command Line Info for Line of Codes by Language*

1.2 Number of Routes Per Service

This table provides a concise overview of the distribution of RESTful API endpoints across all the microservices in the Quiz Portal backend system. Each service—such as User, Quiz, Question, QPFEntry, EmailService, and others—exposes a number of HTTP routes to either serve data or accept input from the client/UI or from other services. These routes are classified by their HTTP methods: GET (used for retrieving data), POST (used for creating or submitting data), and DELETE (used for removing records or users). The table helps illustrate which services are more interaction-heavy and gives an immediate sense of the complexity and size of each microservice based on how many API endpoints they expose. This can be especially useful for assessing testing efforts, reviewing architectural balance, and ensuring maintainability across the service ecosystem.

Table: *Number of Routes Per Service*

Service	GET Routes	POST Routes	DELETE Routes	Total Routes
User	4	5	1	10
Quiz	4	2	0	6
Question	1	2	0	3

QPF Entry	2	2	0	4
Email	0	4	0	4
API Gateway	0	0	0	0 (Proxy only)
Registry	0	0	0	0 (Infra only)

1.3 Route Types and Use Cases

This table provides a functional breakdown of API routes by classifying them according to their HTTP method (GET, POST, DELETE), associated service, and a representative example route. It also includes a brief description of the route's purpose, providing insights into how the backend services contribute to core features of the platform. For instance, GET routes typically handle data retrieval such as user profile info or quiz details, POST routes manage data creation like user sign-up or quiz submissions, and DELETE routes are used for operations like removing users. This table serves as a lightweight form of API documentation, bridging technical implementation with real-world use cases, making it easier for developers and stakeholders to understand how the system functions from both a technical and user perspective.

Table: *Route Types and Use cases*

HTTP Method	Service	Example Routes	Use Case Description
GET	User	/profile, /all-users, /isAdmin, /verifyToken	Fetching user info, verifying role/token
POST	User	/login, /signup, /reset-	User creation, auth, and quiz

		password, /bulk-signup	attempts logging
DELETE	User	/delete-users	Admin removes users
GET	Quiz	/companies, /getAllQuizzesByCompanyName/{name}	Fetching quiz lists, companies
POST	Quiz	/create, /{quizId}/activate	Creating and activating quizzes
GET	Question	/getQuestionsByQuizId/{quizId}	Get all questions in quiz
POST	Question	/create/{quizId}, /bulk-add	Add questions individually or in bulk
GET	QPF Entry	/user/{userId}/quiz/{quizId}, /getDashboardInfo	Get dashboard & quiz progress
POST	QPF Entry	/save	Save quiz results
POST	Email	/send-welcome /send-updatePass, /send-otp, /verify-otp	Email workflows for signup, OTP verification

2. Appendix B: Plagiarism Report

Plagiarism report of the project report.